

Data Structure and Algorithm (PMSCS 623P)

Lecture 16

Graph-Based Algorithms

What is a Graph?

- **Graph** G is a pair (V, E) , where V is a finite set (set of vertices) and E is a finite set of pairs from V (set of edges). We will often denote $n = |V|$, $m = |E|$.
- Graph G can be **directed**, if E consists of ordered pairs, or **undirected**, if E consists of unordered pairs. If $(u, v) \in E$, then vertices u and v are **adjacent**.
- We can assign weight function to the edges: $w_G(e)$ is a weight of edge $e \in E$. The graph which has such function assigned is called **weighted**.
- **Degree** of a vertex v is the number of vertices u for which $(u, v) \in E$ or $(v, u) \in E$ (denote $\deg(v)$). The number of **incoming edges** to a vertex v is called **in-degree** of the vertex (denote $\text{indeg}(v)$). The number of **outgoing edges** from a vertex is called **out-degree** (denote $\text{outdeg}(v)$).

Examples of graphs

Transportation networks How should you design the highway network in a country? What is the quickest way to drive from Badda to Naraynganj?

Communication networks How to send a network packet from Bracu intranet to Yahoo mail server?

Information networks Is the World wide a directed or undirected network?

Social networks Facebook, Myspace, Flickr,

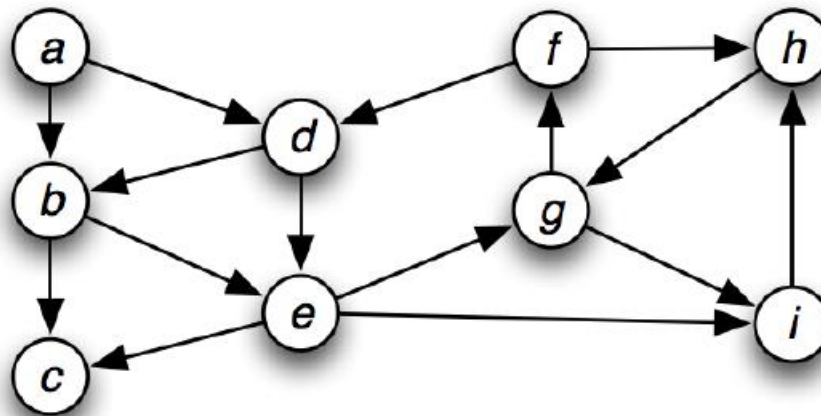
Dependency networks What courses must you take before you can take CSE-423?

What is a Graph?

Definition (Graph)

Graph G is a pair (V, E) , where V is a finite set (set of vertices) and E is a finite set of pairs from V (set of edges). We will often denote $n = |V|$, $m = |E|$.

Example

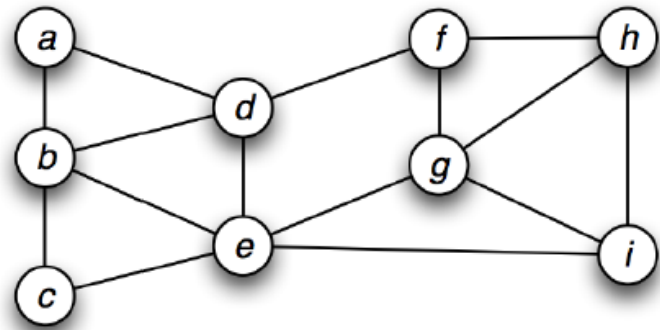
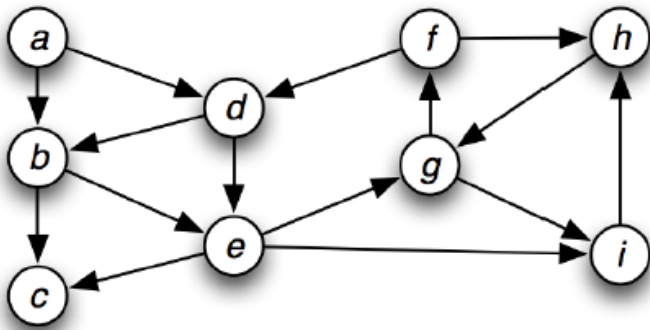


Directed vs. undirected graphs

Definition (Directed vs. undirected graph)

Graph G can be **directed**, if E consists of ordered pairs, or **undirected**, if E consists of unordered pairs. If $(u, v) \in E$, then vertices u and v are **adjacent**.

Example

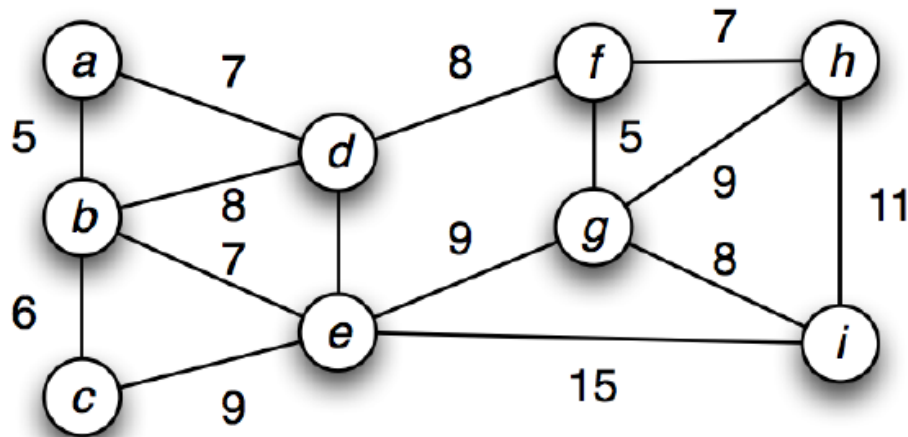


Weighted graphs

Definition (Weighted graph)

We can assign weight function to the edges: $w_G(e)$ is a weight of edge $e \in E$. The graph which has such function assigned is called **weighted**.

Example

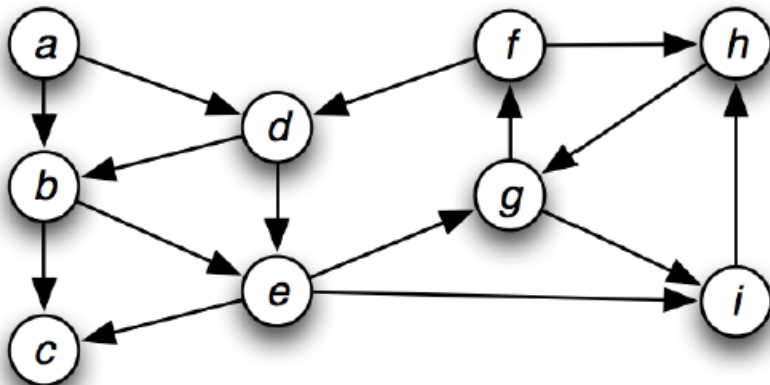


Degree of vertices

Definition (Degree of vertices)

Degree of a vertex v is the number of vertices u for which $(u, v) \in E$ or $(v, u) \in E$ (denote $\deg(v)$). The number of **incoming edges** to a vertex v is called **in-degree** of the vertex (denote $\text{indeg}(v)$). The number of **outgoing edges** from a vertex is called **out-degree** (denote $\text{outdeg}(v)$).

Example (Directed graph)



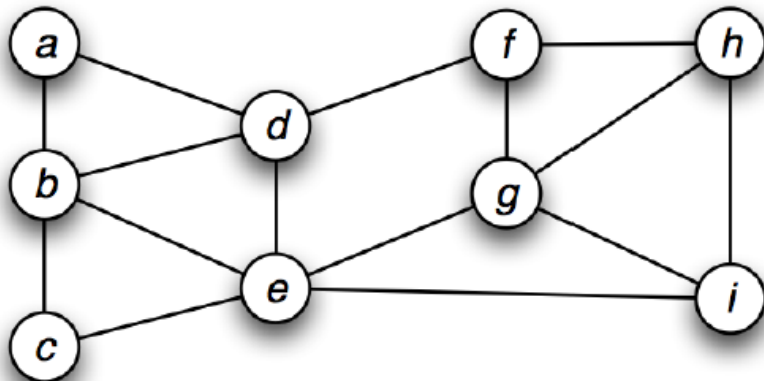
Vertex	in	out
<i>a</i>	0	2
<i>b</i>	2	2
<i>c</i>	2	0
<i>d</i>	2	2
<i>e</i>	2	3

Degree of vertices

Definition (Degree of vertices)

Degree of a vertex v is the number of vertices u for which $(u, v) \in E$ or $(v, u) \in E$ (denote $\deg(v)$). The number of **incoming edges** to a vertex v is called **in-degree** of the vertex (denote $\text{indeg}(v)$). The number of **outgoing edges** from a vertex is called **out-degree** (denote $\text{outdeg}(v)$).

Example (Undirected graph)



Vertex	$\deg$
a	2
b	4
c	2
d	4
e	5

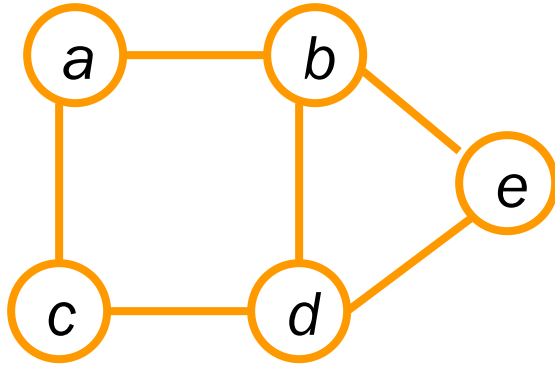
Representing Graphs

Assume $V = \{1, 2, \dots, n\}$

An adjacency matrix represents the graph as a $n \times n$ matrix A :

- $A[i, j]$ = 1 if edge $(i, j) \in E$ (or weight of edge)
= 0 if edge $(i, j) \notin E$

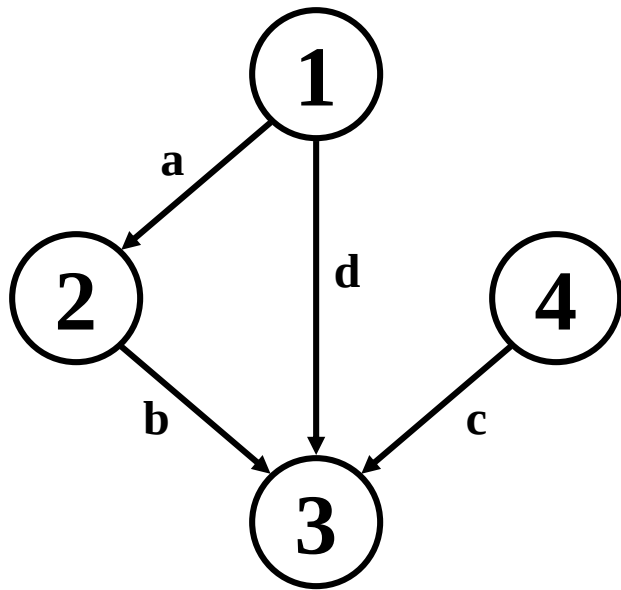
Adjacency Matrix Representation



	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>a</i>	0	1	1	0	0
<i>b</i>	1	0	0	1	1
<i>c</i>	1	0	0	1	0
<i>d</i>	0	1	1	0	1
<i>e</i>	0	1	0	1	0

Graphs: Adjacency Matrix

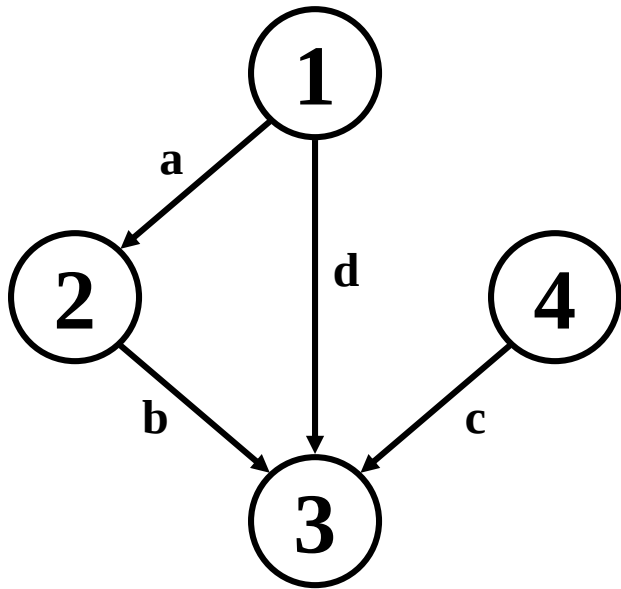
Example:



A	1	2	3	4
1				
2				
3			??	
4				

Graphs: Adjacency Matrix

Example:



A	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	0	0	1	0

Adjacency Matrix Representation

Memory required

- $O(V^2)$

Preferred when

- The graph is **dense**: $E = O(V^2)$

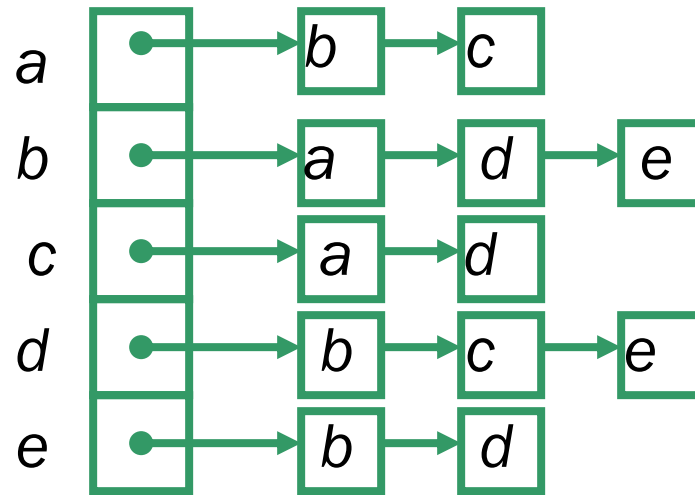
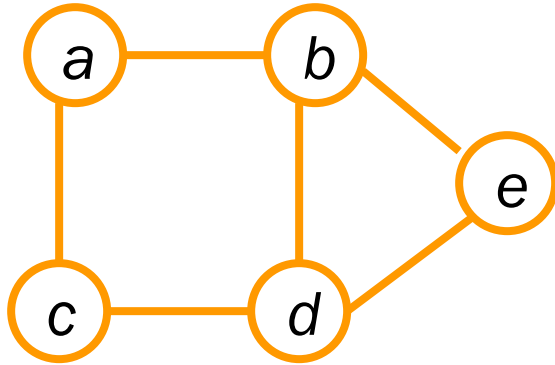
Advantage

- Can quickly determine if there is an edge between two vertices

Disadvantage

- No quick way to determine the vertices adjacent from another vertex.
- Listing all the neighbors of ANY vertex takes $O(V)$ time.

Adjacency List Representation



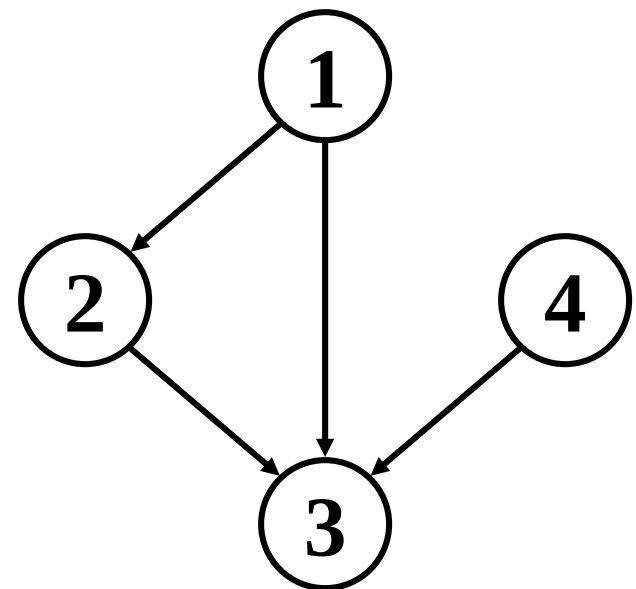
Graphs: Adjacency List

Adjacency list: for each vertex $v \in V$, store a list of vertices adjacent to v

Example:

- $\text{Adj}[1] = \{2, 3\}$
- $\text{Adj}[2] = \{3\}$
- $\text{Adj}[3] = \{\}$
- $\text{Adj}[4] = \{3\}$

Variation: can also keep
a list of edges coming into vertex



Adjacency List Representation

Memory required

- $O(V + E)$

Preferred when

$O(V)$ for sparse graphs since $E=O(V)$

- for **sparse** graphs: $E = O(V)$ $O(V^2)$ for dense graphs since $E=O(V^2)$

Disadvantage

- No quick way to determine whether there is an edge between vertices u and v

Advantage

- Can quickly determine all the vertices adjacent from a given vertex, v : takes $O(N(v))$ time, where $N(v)$ is the number of neighbors of v .

Graph Searching

Given: a graph $G = (V, E)$, directed or undirected

Goal: methodically explore every vertex and every edge

Ultimately: build a tree on the graph

- Pick a vertex as the root
- Choose certain edges to produce a tree
- Note: might also build a *forest* if graph is not connected

Graph Traversals

Ways to traverse/search a graph

- Visit every vertex exactly once
- Breadth-First Search
- Depth-First Search

Breadth-First Search

“Explore” a graph, turning it into a tree

- One vertex at a time
- Expand frontier of explored vertices across the *breadth* of the frontier

Builds a tree over the graph

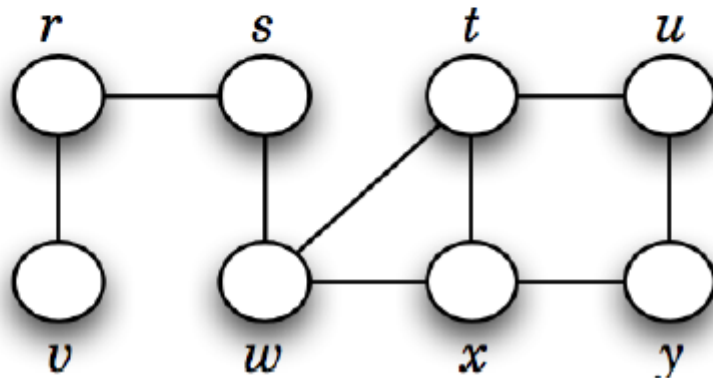
- Pick a *source vertex* to be the root
- Find (“discover”) all of its children, then their children, etc.

Basic idea behind breadth first search (BFS)

Given a undirected graph $G = (V, E)$, and a distinguished vertex s , systematically visit each vertex $v \in V$ that is reachable from s .

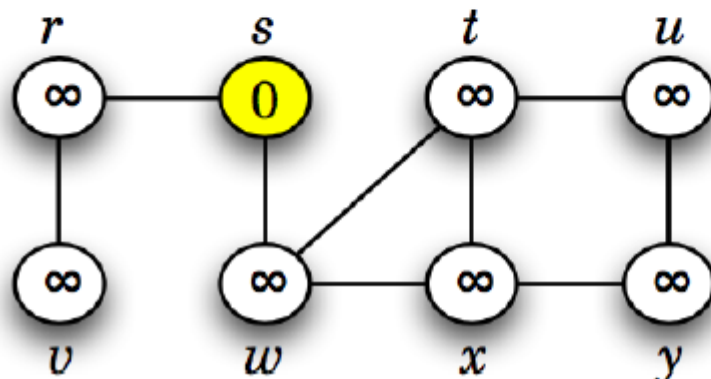
Breadth first search (BFS)

```
BFS( $G, s$ )   $\triangleright G = (V, E)$   
1   $Q \leftarrow \{s\}$   
2   $d[v] \leftarrow \infty, \forall v \in V; d[s] \leftarrow 0$   
3  while  $Q \neq \emptyset$   
4      do  $u \leftarrow \text{DEQUEUE}(Q)$   
5          for each  $v \in \text{Adj}[u]$   
6              do if  $d[v] = \infty$   
7                  then  $d[v] = d[u] + 1$   
8                       $\text{ENQUEUE}(Q, v)$ 
```



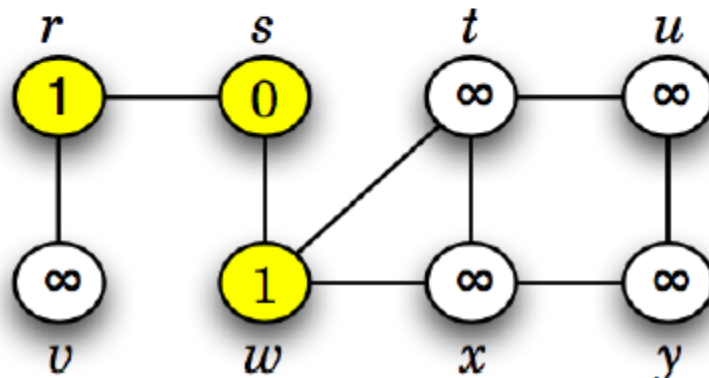
Breadth first search (BFS)

```
BFS( $G, s$ )   $\triangleright G = (V, E)$   
1   $Q \leftarrow \{s\}$   
2   $d[v] \leftarrow \infty, \forall v \in V; d[s] \leftarrow 0$   
3  while  $Q \neq \emptyset$   
4      do  $u \leftarrow \text{DEQUEUE}(Q)$   
5          for each  $v \in \text{Adj}[u]$   
6              do if  $d[v] = \infty$   
7                  then  $d[v] = d[u] + 1$   
8                       $\text{ENQUEUE}(Q, v)$ 
```



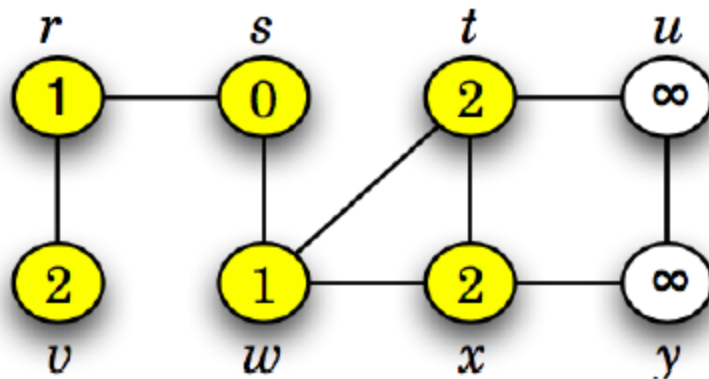
Breadth first search (BFS)

```
BFS( $G, s$ )   $\triangleright G = (V, E)$   
1   $Q \leftarrow \{s\}$   
2   $d[v] \leftarrow \infty, \forall v \in V; d[s] \leftarrow 0$   
3  while  $Q \neq \emptyset$   
4      do  $u \leftarrow \text{DEQUEUE}(Q)$   
5          for each  $v \in \text{Adj}[u]$   
6              do if  $d[v] = \infty$   
7                  then  $d[v] = d[u] + 1$   
8                       $\text{ENQUEUE}(Q, v)$ 
```



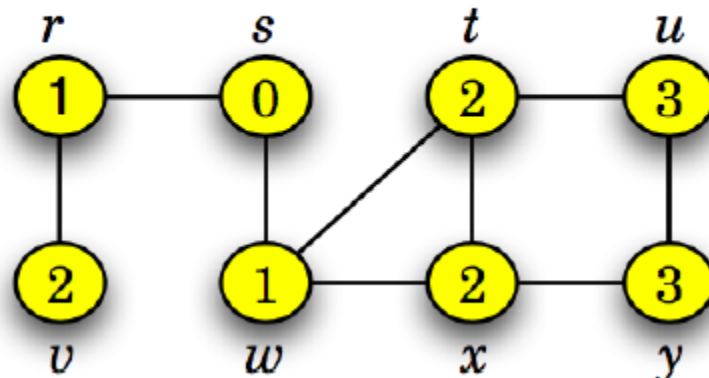
Breadth first search (BFS)

```
BFS( $G, s$ )   $\triangleright G = (V, E)$   
1   $Q \leftarrow \{s\}$   
2   $d[v] \leftarrow \infty, \forall v \in V; d[s] \leftarrow 0$   
3  while  $Q \neq \emptyset$   
4      do  $u \leftarrow \text{DEQUEUE}(Q)$   
5          for each  $v \in \text{Adj}[u]$   
6              do if  $d[v] = \infty$   
7                  then  $d[v] = d[u] + 1$   
8                       $\text{ENQUEUE}(Q, v)$ 
```

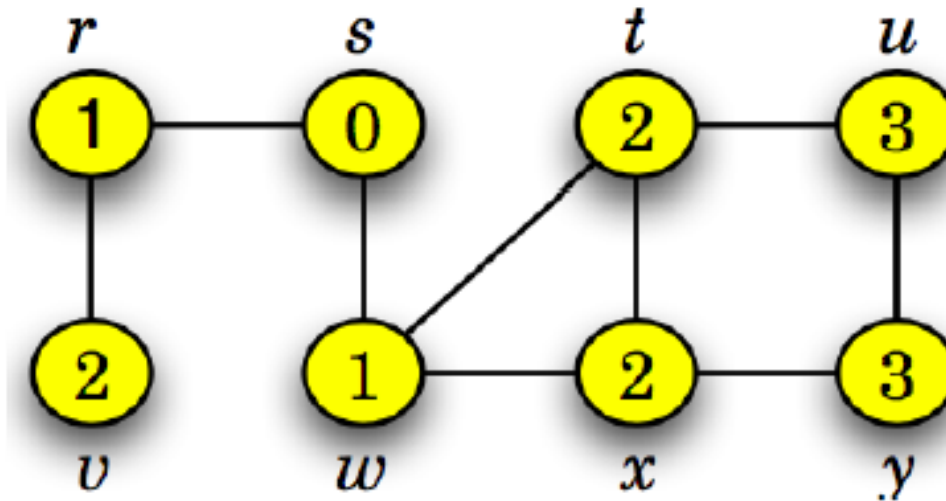


Breadth first search (BFS)

```
BFS( $G, s$ )  ▷  $G = (V, E)$   
1   $Q \leftarrow \{s\}$   
2   $d[v] \leftarrow \infty, \forall v \in V; d[s] \leftarrow 0$   
3  while  $Q \neq \emptyset$   
4      do  $u \leftarrow \text{DEQUEUE}(Q)$   
5          for each  $v \in \text{Adj}[u]$   
6              do if  $d[v] = \infty$   
7                  then  $d[v] = d[u] + 1$   
8                       $\text{ENQUEUE}(Q, v)$ 
```



What does *this* $\text{BFS}(G, s)$ algorithm tell us?



- This graph $G = (V, E)$ is connected since BFS visits all vertices $v \in V$ starting from s .
- The minimum number of *hops* or edges between s and any vertex $t \in V$.

A better Breadth first search (BFS) algorithm

Information during traversal

- ① $color[v]$: the **color** of each vertex visited
 - white: **undiscovered**
 - gray: **discovered** but not finished processing
 - black: **finished** processing
- ② $\pi[v]$: the **predecessor** pointer
- ③ $d[v]$: the **discovery time** (or *hops* from the start)

Breadth-First Search

- **Input:** Graph $G = (V, E)$, either directed or undirected, and *source vertex* $s \in V$.
- **Output:**
 - $d[v]$ = distance (smallest # of edges, or shortest path) from s to v , for all $v \in V$. $d[v] = \infty$ if v is not reachable from s .
 - $\pi[v] = u$ such that (u, v) is last edge on shortest path $s \rightsquigarrow v$.
 - u is v 's **predecessor**.
 - Builds breadth-first tree with root s that contains all reachable vertices.

Breadth-First Search

Associate vertex “colors” to guide the algorithm

- White vertices have not been discovered
 - All vertices start out white
- Grey vertices are discovered but not fully explored
 - They may be adjacent to white vertices
- Black vertices are discovered and fully explored
 - They are adjacent only to black and gray vertices

Explore vertices by scanning adjacency list of grey vertices

BFS Trees

- ❑ BFS tree is not necessarily unique for a given graph
- ❑ Depends on the order in which neighboring vertices are processed
- ❑ During the breadth-first search, assign an integer to each vertex
- ❑ Indicate the distance of each vertex from the source s

BFS(G, s)

```
1.   for each vertex  $u$  in  $V[G] - \{s\}$ 
2.        $color[u] \leftarrow white$ 
3.        $d[u] \leftarrow \infty$ 
4.        $\pi[u] \leftarrow nil$ 
5.    $color[s] \leftarrow gray$ 
6.    $d[s] \leftarrow 0$ 
7.    $\pi[s] \leftarrow nil$ 
8.    $Q \leftarrow \Phi$ 
9.   enqueue( $Q, s$ )
10.  while  $Q \neq \Phi$ 
11.       $u \leftarrow dequeue(Q)$ 
12.      for each  $v$  in  $Adj[u]$ 
13.          if  $color[v] = white$ 
14.               $color[v] \leftarrow gray$ 
15.               $d[v] \leftarrow d[u] + 1$ 
16.               $\pi[v] \leftarrow u$ 
17.              enqueue( $Q, v$ )
18.       $color[u] \leftarrow black$ 
```

initialization

access source s

white: undiscovered
gray: discovered
black: finished

Q : a queue of discovered vertices
 $color[v]$: color of v
 $d[v]$: distance from s to v
 $\pi[u]$: predecessor of v

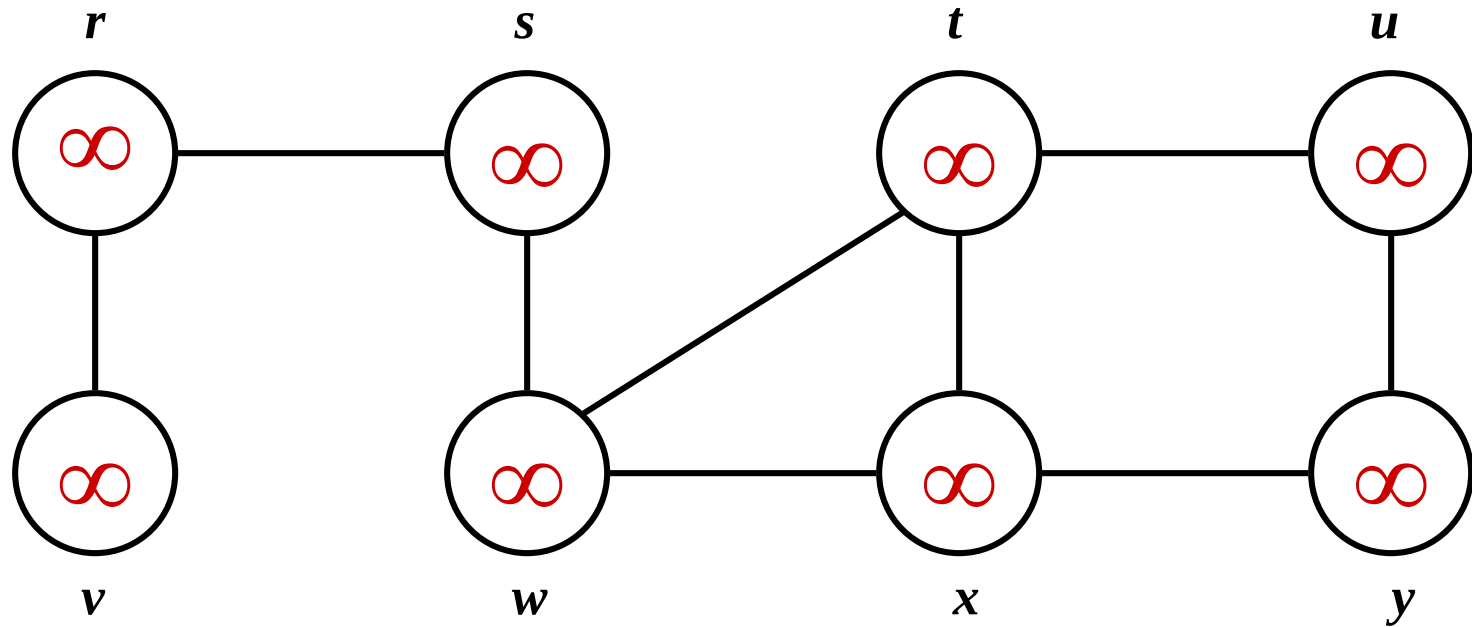
A better Breadth first search (BFS) algorithm

$\text{BFS}(G, s) \triangleright G = (V, E)$

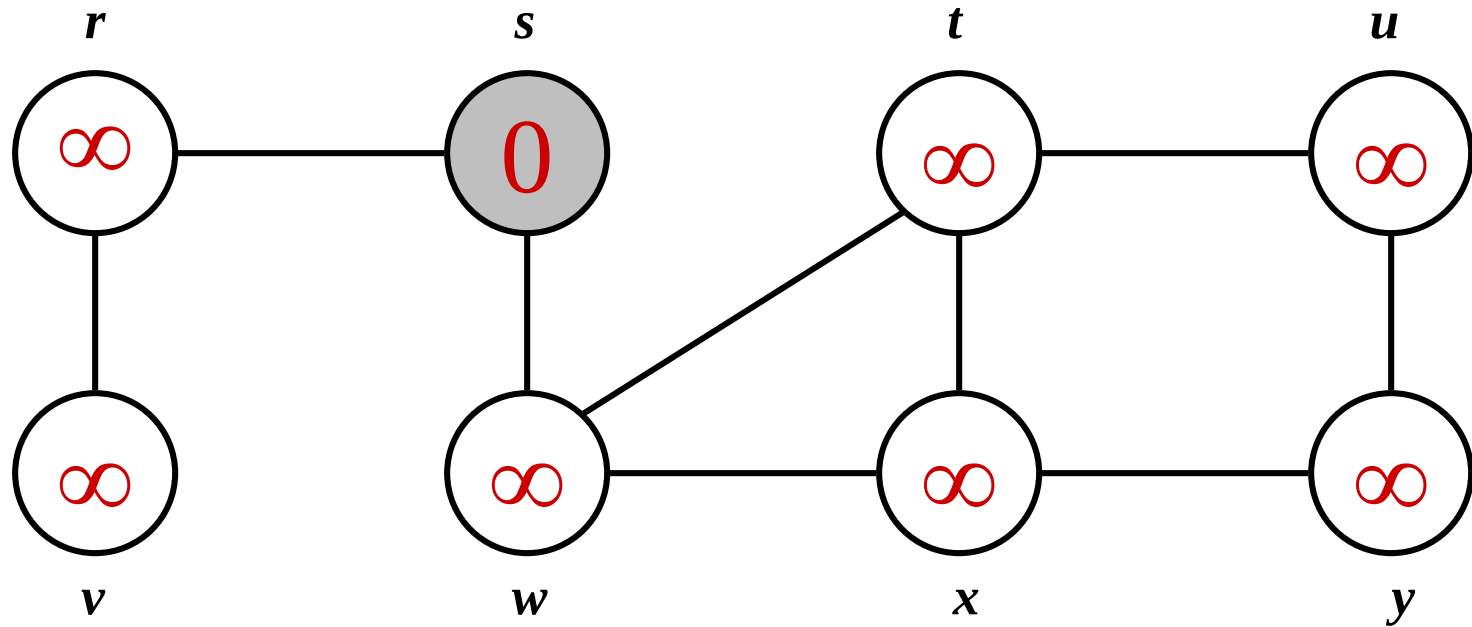
$\triangleright$ Initialize the vertex colors and predecessors

```
1 for each vertex  $v \in V$ 
2     do  $color[v] = \text{WHITE}$ 
3        $\pi[v] = \text{NULL}$ 
4        $d[v] = \infty$ 
5  $color[s] = \text{GRAY}; \pi[s] = \text{NULL}; d[s] = 0$ 
6  $Q = \emptyset; \text{ENQUEUE}(Q, s)$ 
7 while  $Q \neq \emptyset$ 
8     do  $u = \text{DEQUEUE}(Q)$ 
9       for each vertex  $v \in \text{Adj}[u]$ 
10          do if  $color[v] = \text{WHITE}$ 
11             then  $color[v] = \text{GRAY}$ 
12                   $d[v] = d[u] + 1$ 
13                   $\pi[v] = u$ 
14                   $\text{ENQUEUE}(Q, v)$ 
15      $color[u] = \text{BLACK}$ 
```


Breadth-First Search: Example

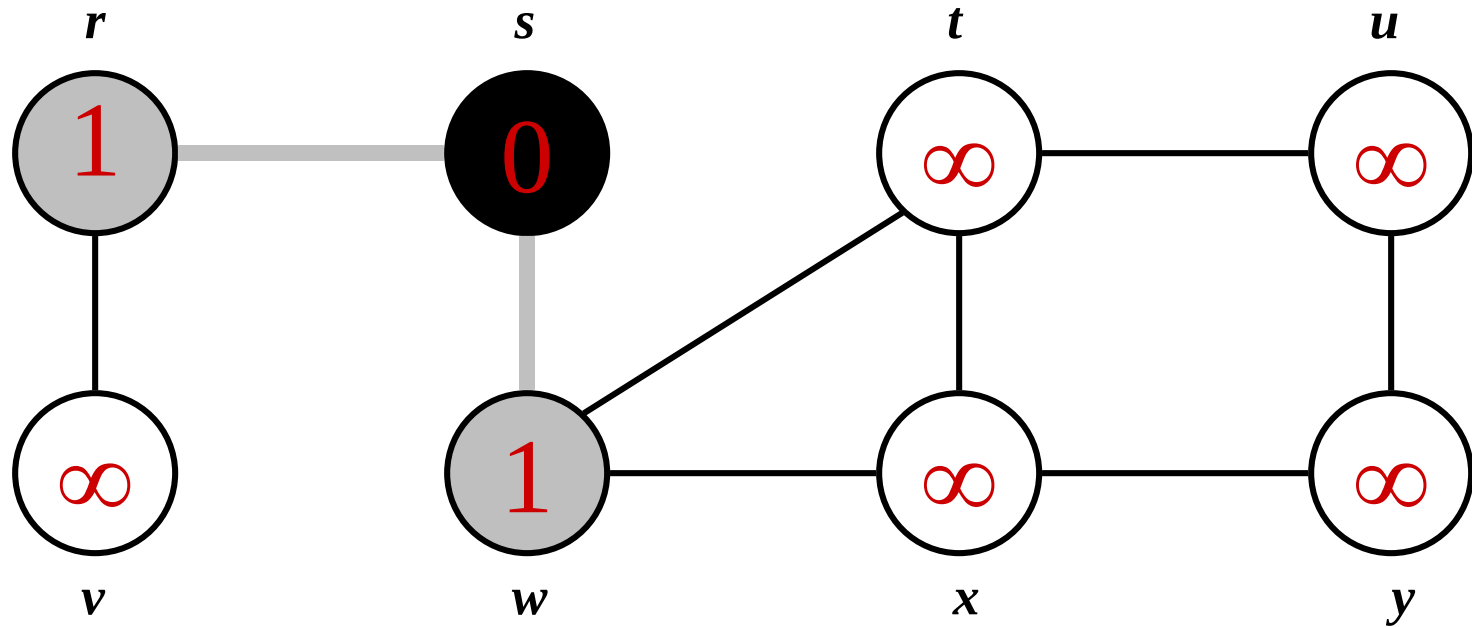


Breadth-First Search: Example



$Q:$ s

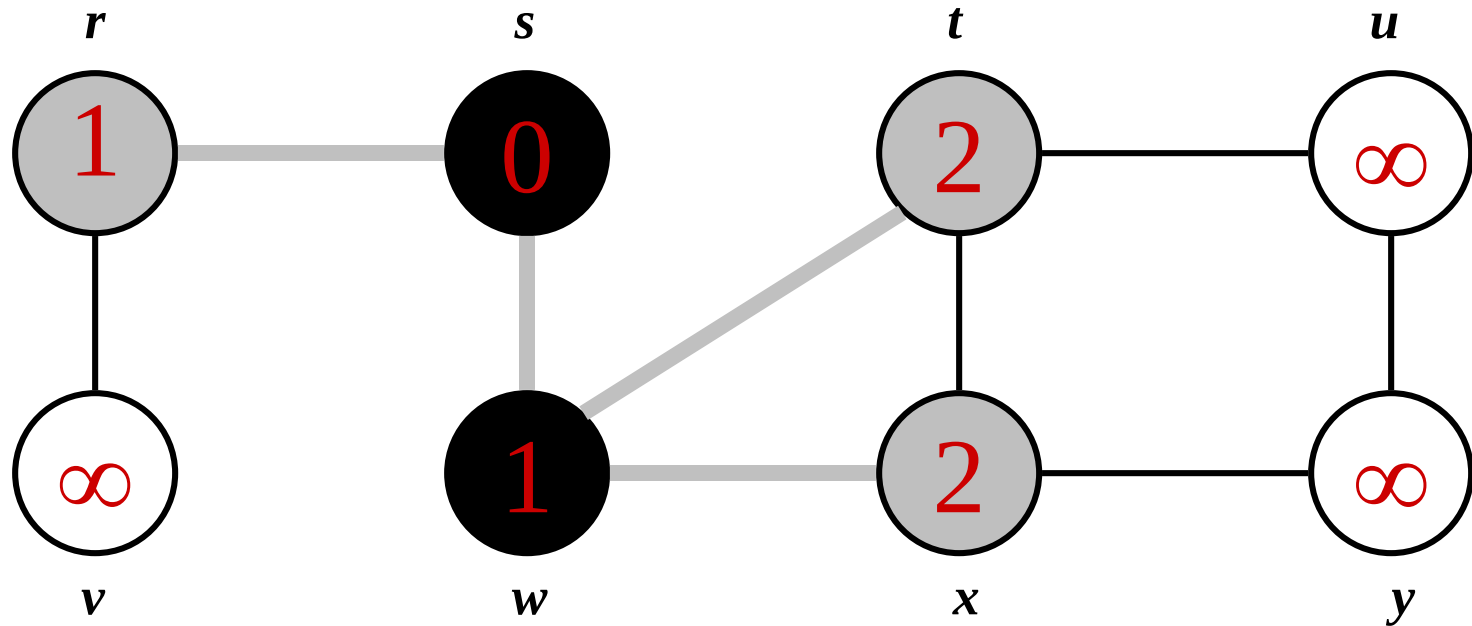
Breadth-First Search: Example



Q :

w	r
-----	-----

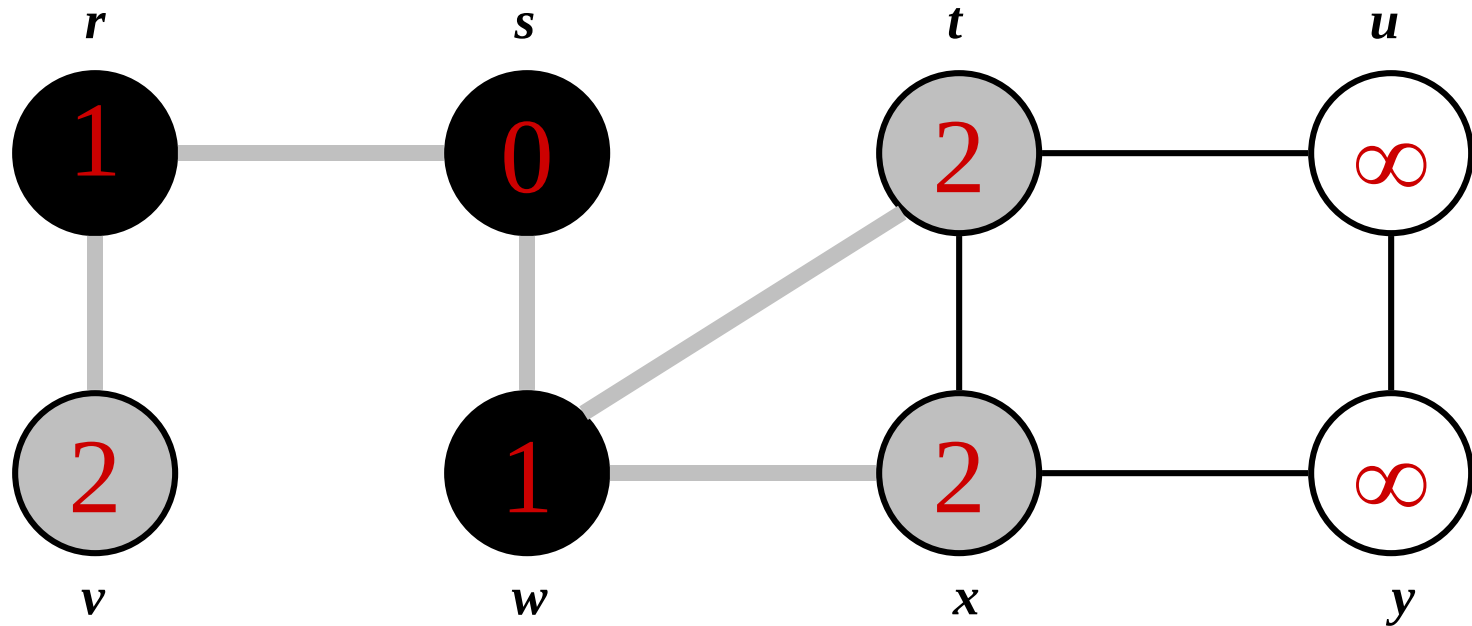
Breadth-First Search: Example



Q :

r	t	x
-----	-----	-----

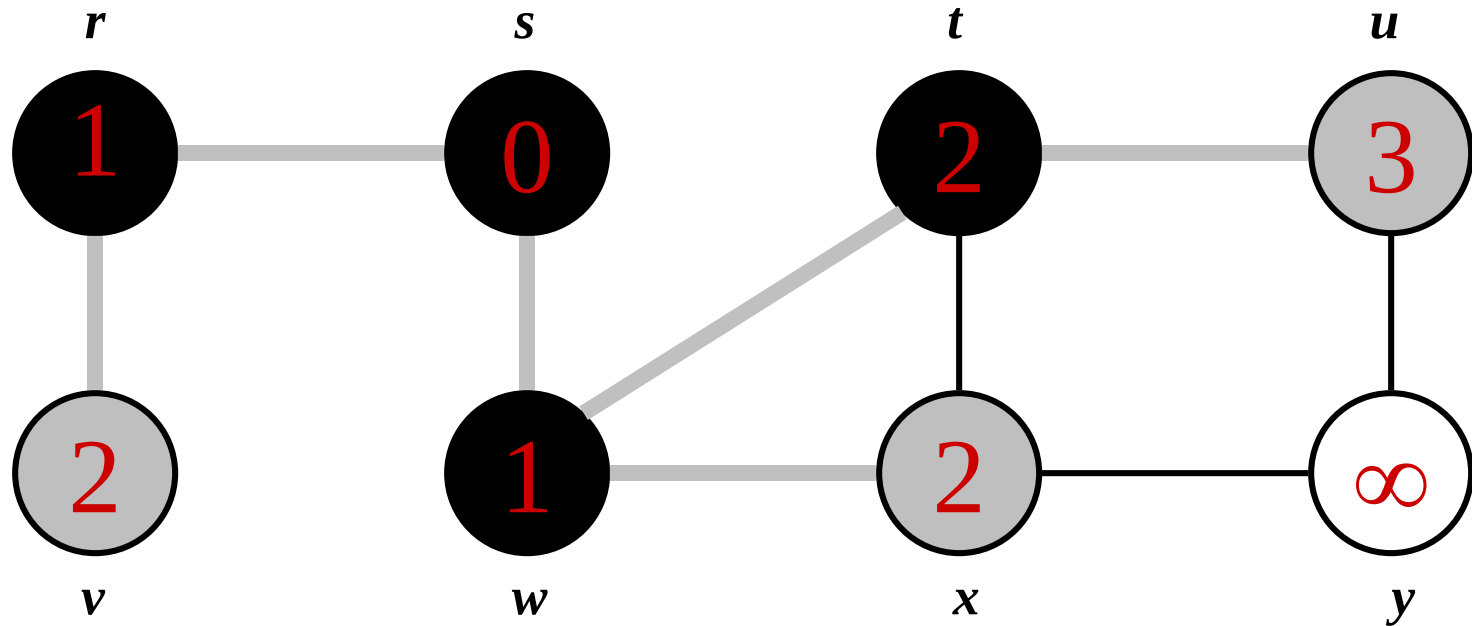
Breadth-First Search: Example



Q :

t	x	v
-----	-----	-----

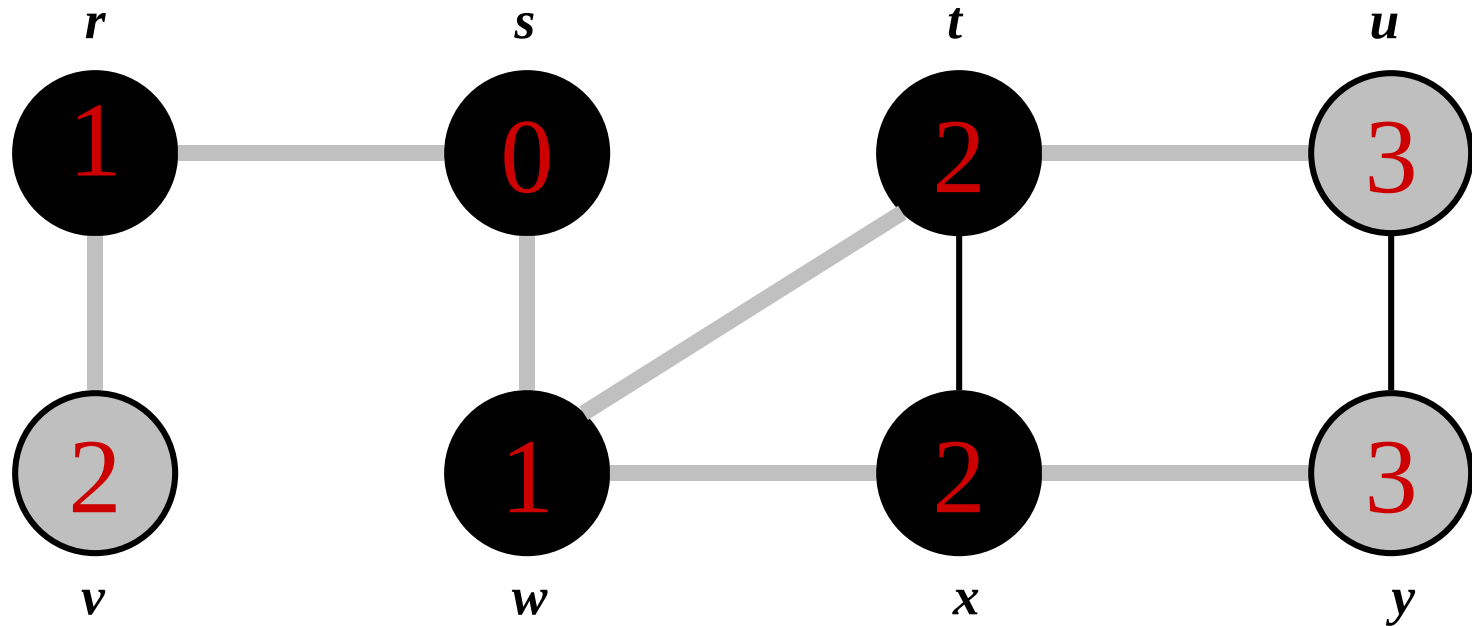
Breadth-First Search: Example



Q :

x	v	u
-----	-----	-----

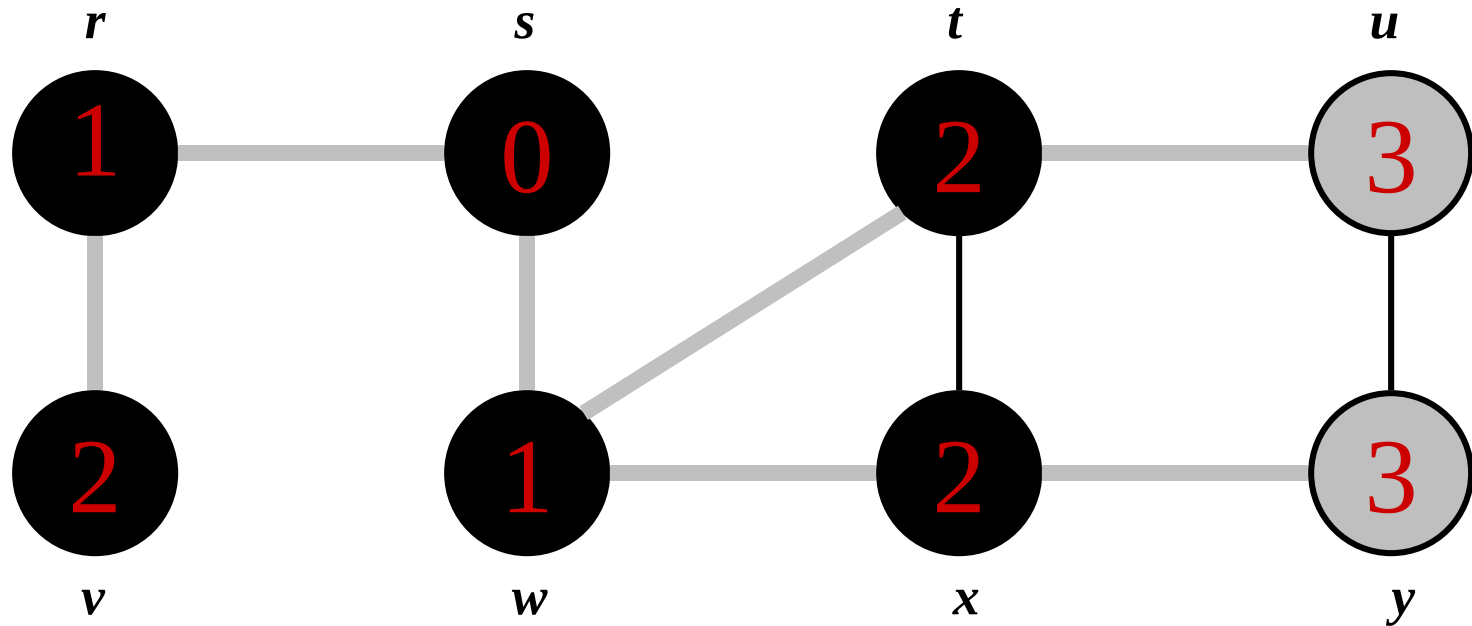
Breadth-First Search: Example



Q:

v	u	y
-----	-----	-----

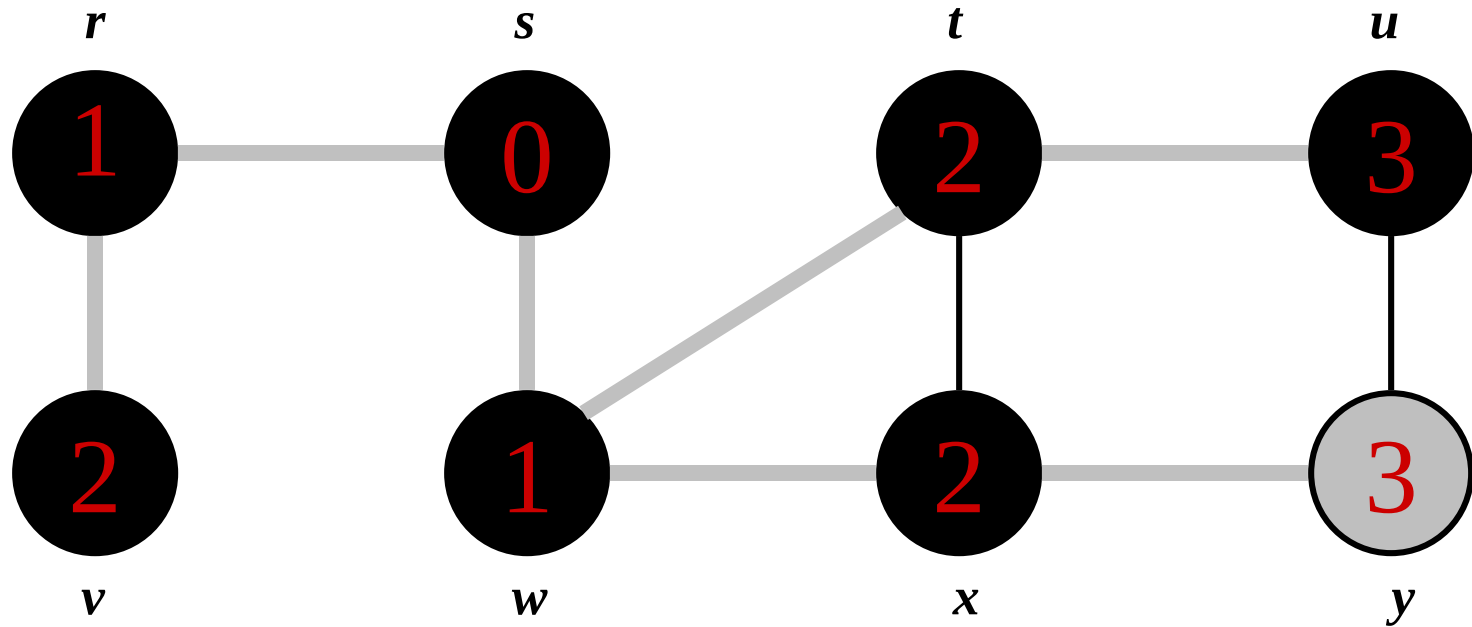
Breadth-First Search: Example



Q:

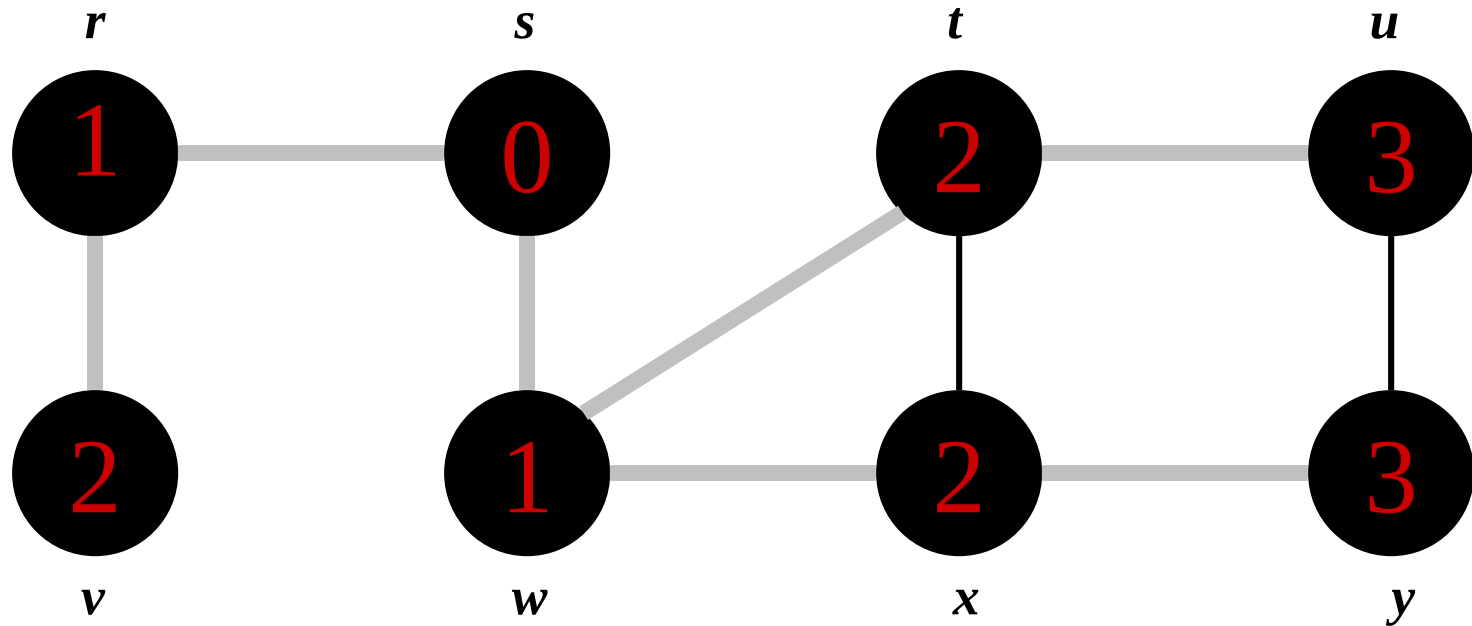
u	y
-----	-----

Breadth-First Search: Example

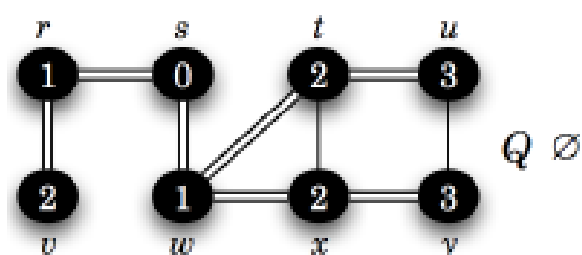
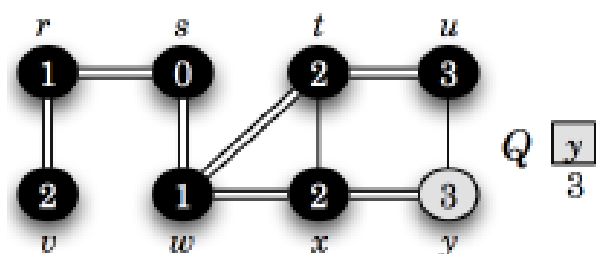
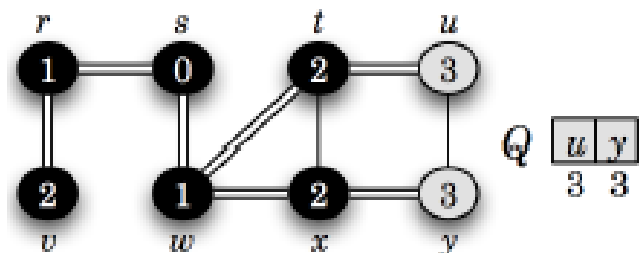
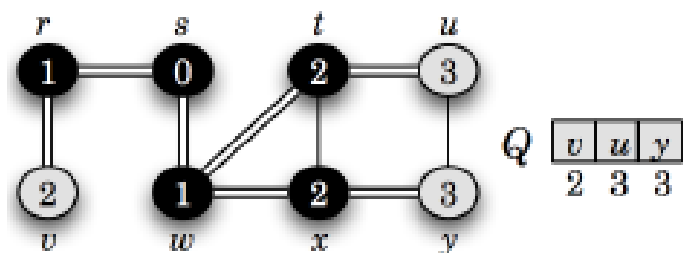
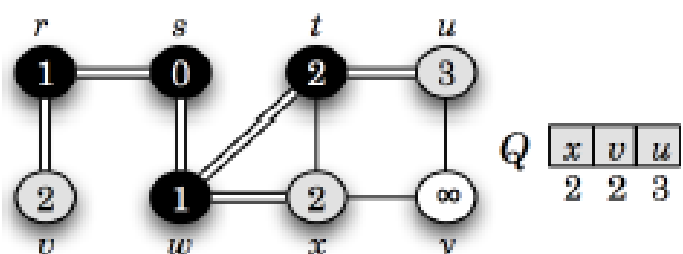
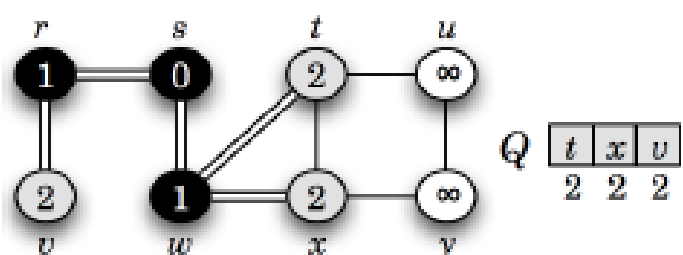
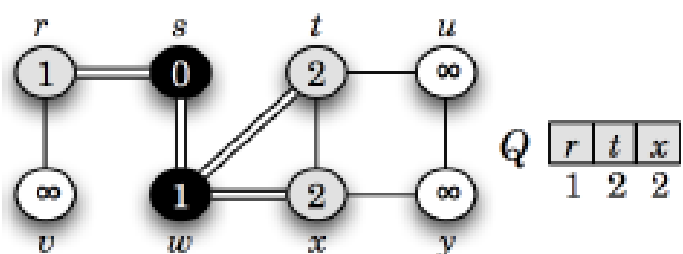
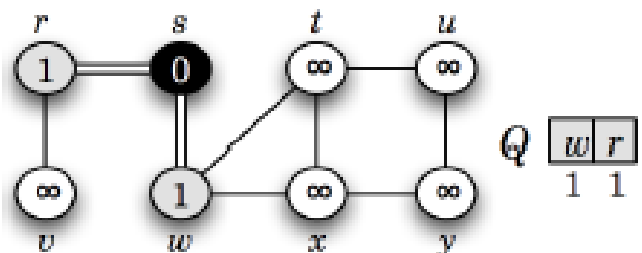
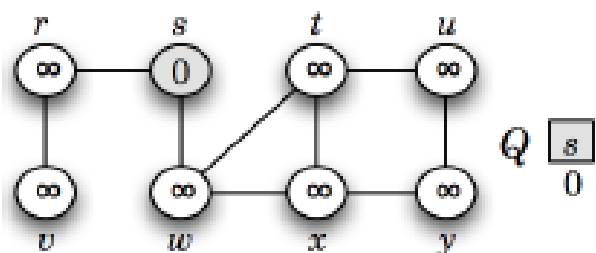


Q : y

Breadth-First Search: Example



$Q: \emptyset$



BFS Running Time

Initialization of each vertex takes $O(V)$ time

Every vertex is enqueued once and dequeued once, taking $O(V)$ time

When a vertex is dequeued, all its neighbors are checked to see if they are unvisited, taking time proportional to number of neighbors of the vertex, and summing to $O(E)$ over all iterations

Total time is $O(V+E)$

Breadth-First Search: Properties

BFS calculates the *shortest-path distance* to the source node

- Shortest-path distance $\delta(s,v)$ = minimum number of edges from s to v , or ∞ if v not reachable from s

BFS builds *breadth-first tree*, in which paths to root represent shortest paths in G

- Thus we can use BFS to calculate shortest path from one vertex to another in $O(V+E)$ time

Depth-First Search

Depth-first search is another strategy for exploring a graph

- Explore “deeper” in the graph whenever possible
- Edges are explored out of the most recently discovered vertex v that still has unexplored edges
- When all of v 's edges have been explored, backtrack to the vertex from which v was discovered

Depth-first Search (DFS)

- Explore edges out of the most recently discovered vertex v .
- When all edges of v have been explored, backtrack to explore other edges leaving the vertex from which v was discovered (its *predecessor*).
- “Search as deep as possible first.”
- Continue until all vertices reachable from the original source are discovered.
- If any undiscovered vertices remain, then one of them is chosen as a new source and search is repeated from that source.

Depth First Search

- Starts from an initial vertex.
- Recursively visits each adjacent vertex.
- Strategy to search **deeper** into the graph, unlike BFS which fans out.

Information during traversal

- 1 $color[v]$: the **color** of each vertex visited
 - white: **undiscovered**
 - gray: **discovered** but not finished processing
 - black: **finished** processing
- 2 $\pi[v]$: the **predecessor** pointer
- 3 $d[v]$: the **discovery time** (or *hops* from the start)
- 4 $f[v]$: the **finish time** – when processing of v and all its descendants have finished.

Depth-first Search

- **Input:** $G = (V, E)$, directed or undirected.
No source vertex given!
- **Output:**
 - 2 **timestamps** on each vertex.
 - $d[v]$ = *discovery time* (v turns from white to gray)
 - $f[v]$ = *finishing time* (v turns from gray to black)
 - $\pi[v]$: predecessor of $v = u$, such that v was discovered during the scan of u 's adjacency list.
 - Depth-first forest

Depth-first Search (Pseudocode)

DFS(G)

1. **for** each vertex $u \in V[G]$
2. **do** $color[u] \leftarrow \text{white}$
3. $\pi[u] \leftarrow \text{NIL}$
4. $time \leftarrow 0$
5. **for** each vertex $u \in V[G]$
6. **do if** $color[u] = \text{white}$
7. **then** DFS-Visit(u)

Uses a global timestamp **time**.

DFS-Visit(u)

1. $color[u] \leftarrow \text{GRAY}$ // White vertex u
 has been discovered
2. $time \leftarrow time + 1$
3. $d[u] \leftarrow time$
4. **for** each $v \in Adj[u]$
5. **do if** $color[v] = \text{WHITE}$
6. **then** $\pi[v] \leftarrow u$
7. DFS-Visit(v)
8. $color[u] \leftarrow \text{BLACK}$ // Blacken u ;
 it is finished.
9. $f[u] \leftarrow time \leftarrow time + 1$

Depth-first Search (Pseudocode)

$\text{DFS}(G) \triangleright G = (V, E)$

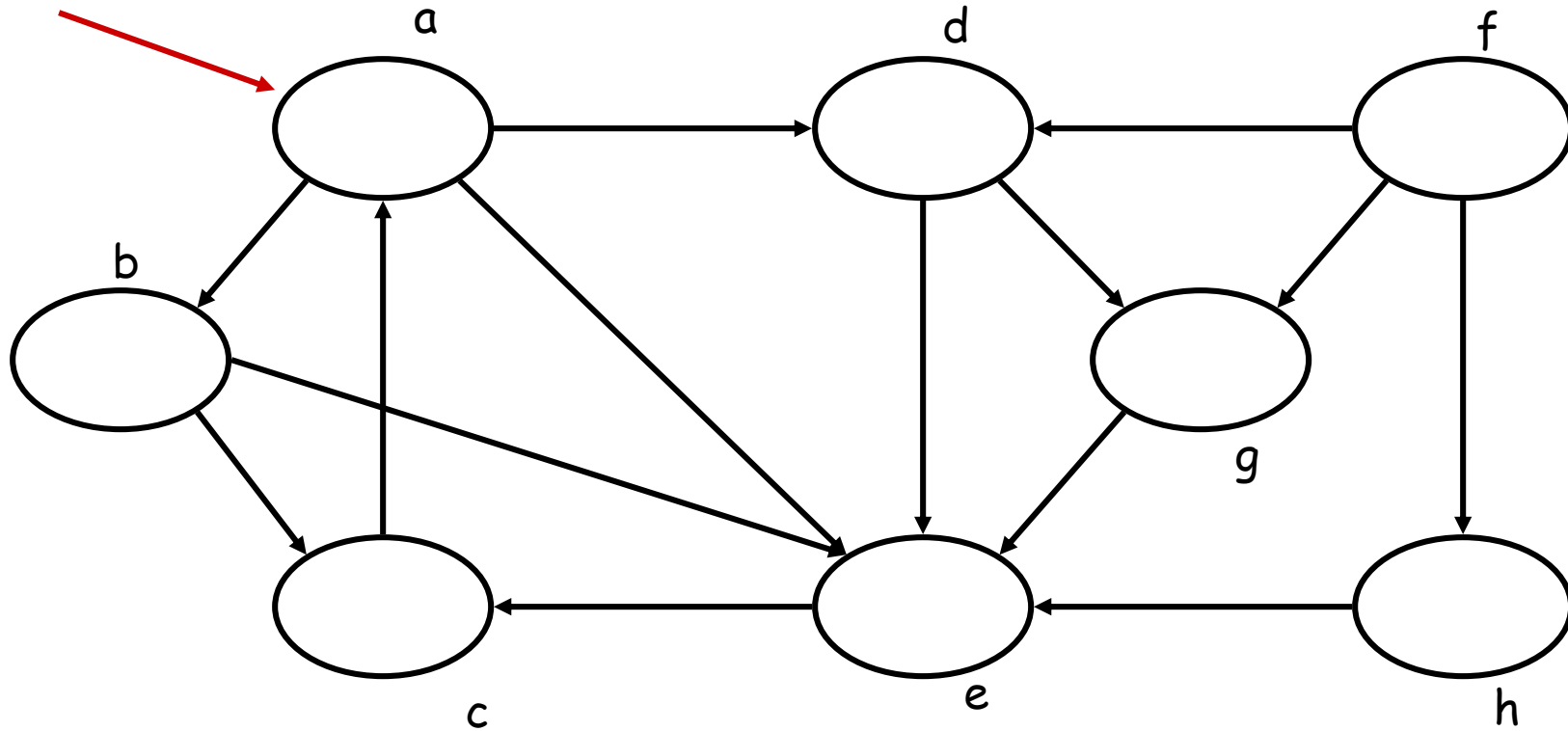
```
1  for each vertex  $u \in V[G]$ 
2      do  $\text{color}[u] \leftarrow \text{WHITE}$ 
3      do  $\pi[u] \leftarrow \text{NIL}$ 
4   $\text{time} \leftarrow 0$ 
5  for each vertex  $u \in V[G]$ 
6      do if  $\text{color}[u] = \text{WHITE}$ 
7          then  $\text{DFS-VISIT}(u)$ 
```

$\text{DFS-VISIT}(G, u) \triangleright G = (V, E), u \in V$

```
1   $\text{color}[u] \leftarrow \text{GRAY}$ 
2   $\text{time} \leftarrow \text{time} + 1$ 
3   $d[u] \leftarrow \text{time}$ 
4  for each vertex  $v \in \text{Adj}[u]$ 
5      do if  $\text{color}[v] = \text{WHITE}$ 
6          then  $\pi[v] \leftarrow u$ 
7               $\text{DFS-VISIT}(v)$ 
8   $\text{color}[u] \leftarrow \text{BLACK}$ 
9   $f[u] \leftarrow \text{time} + 1$ 
```

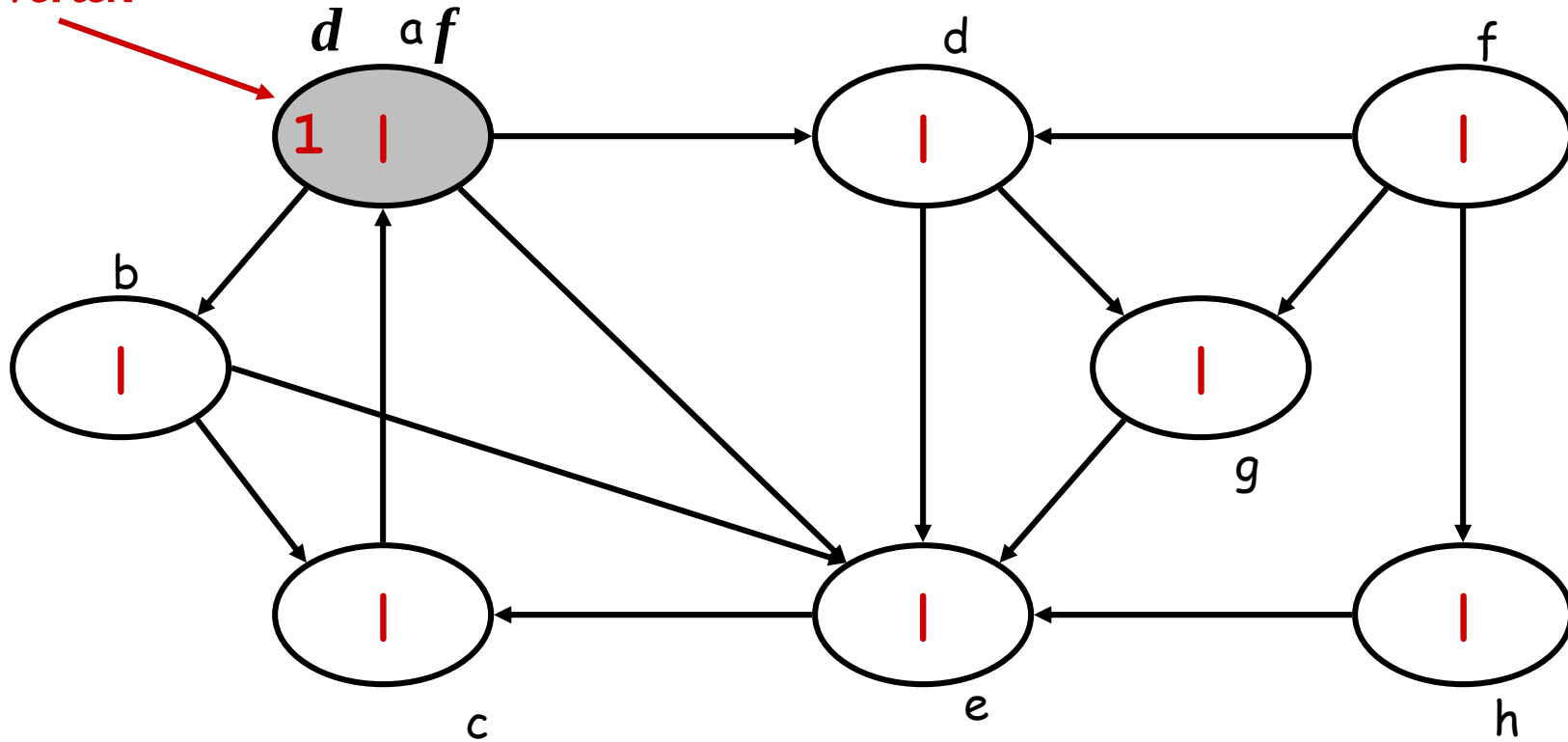
DFS Example

**source
vertex**



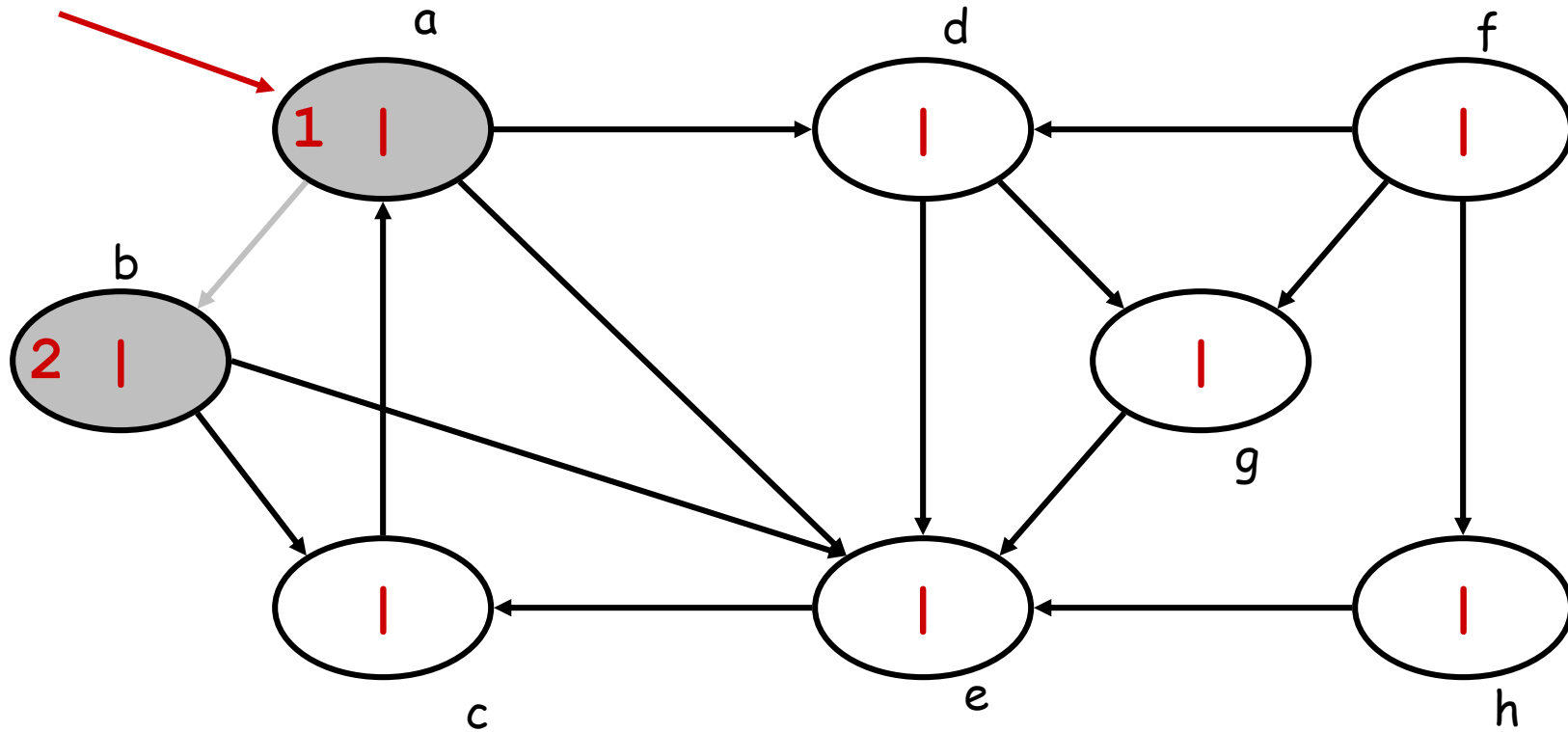
DFS Example

*source
vertex*



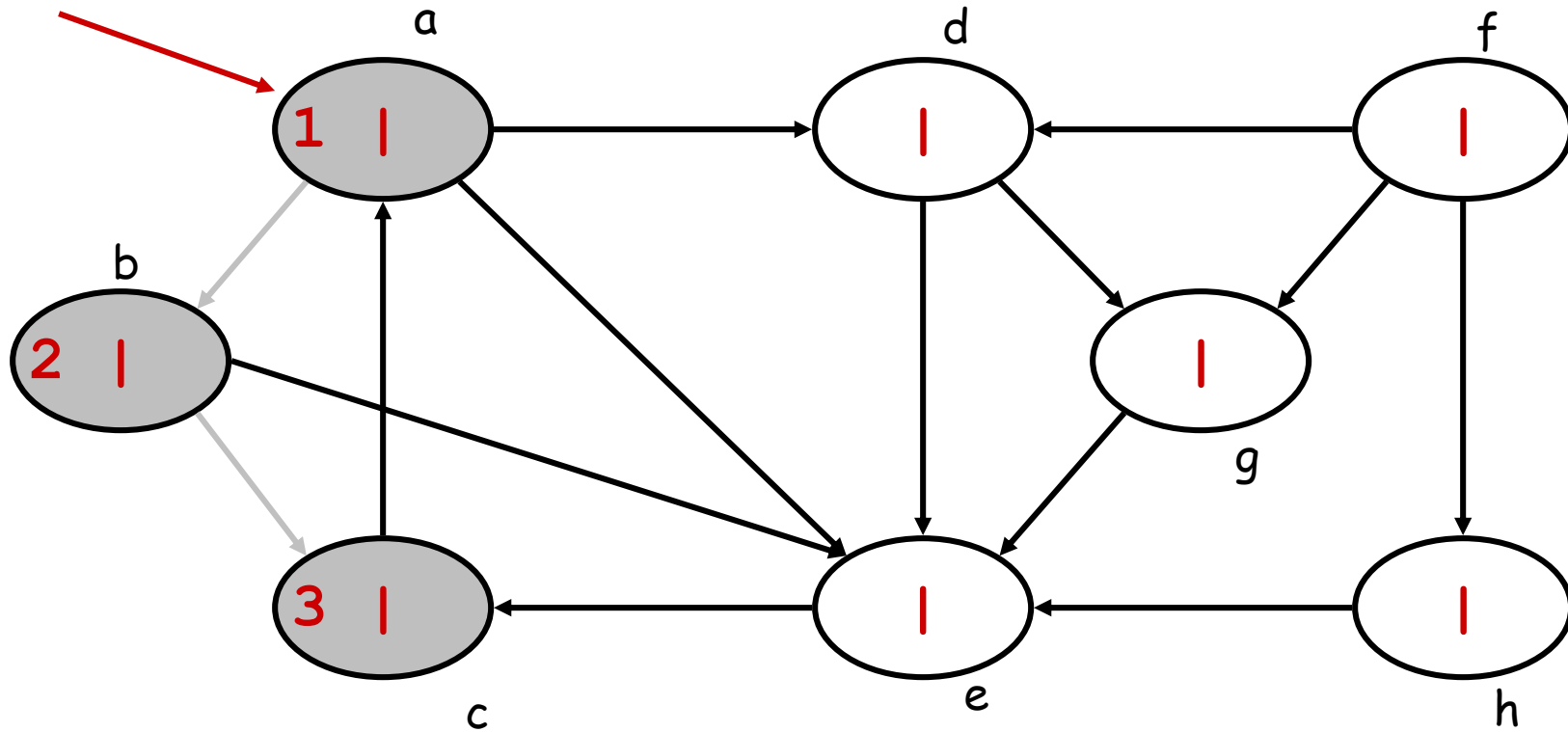
DFS Example

*source
vertex*



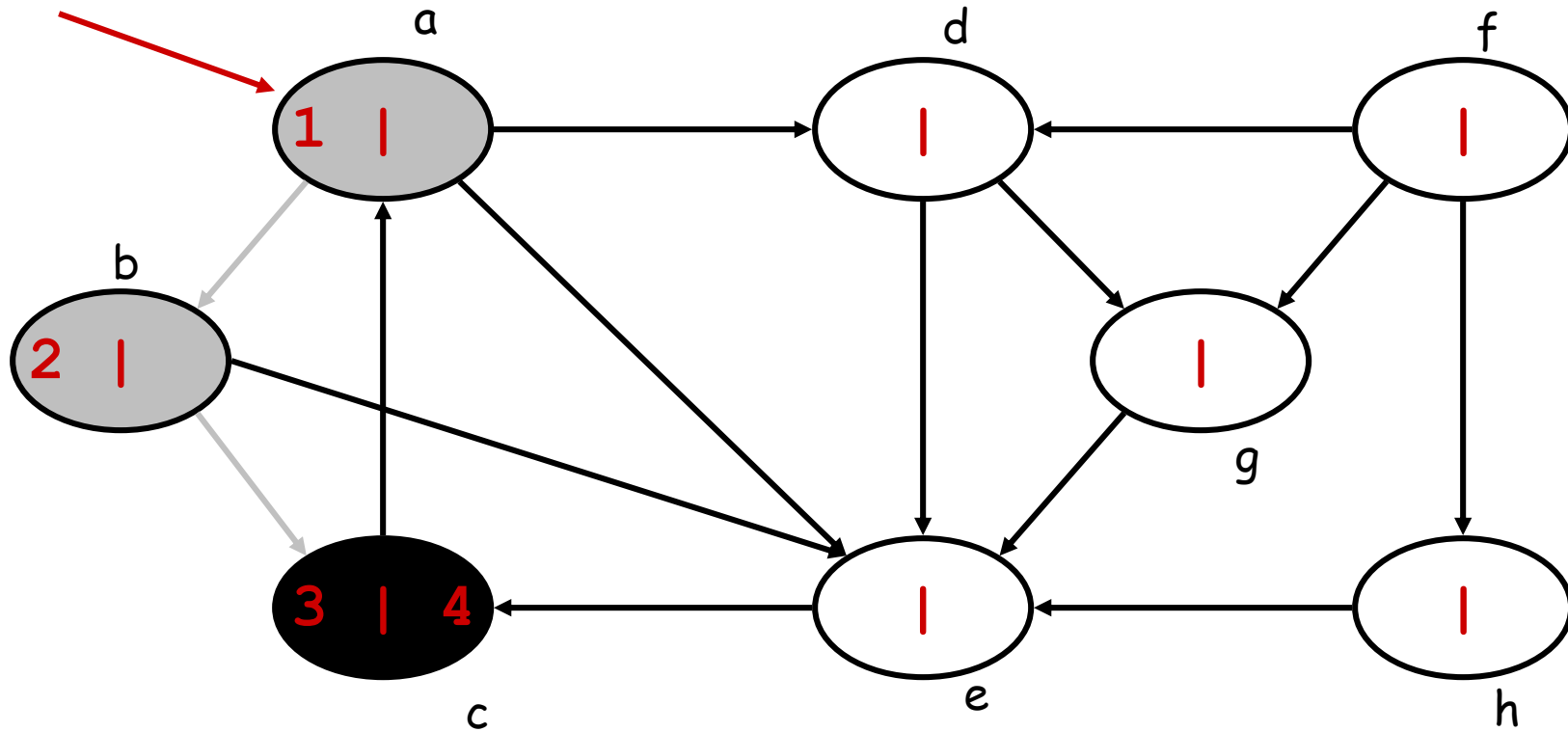
DFS Example

*source
vertex*



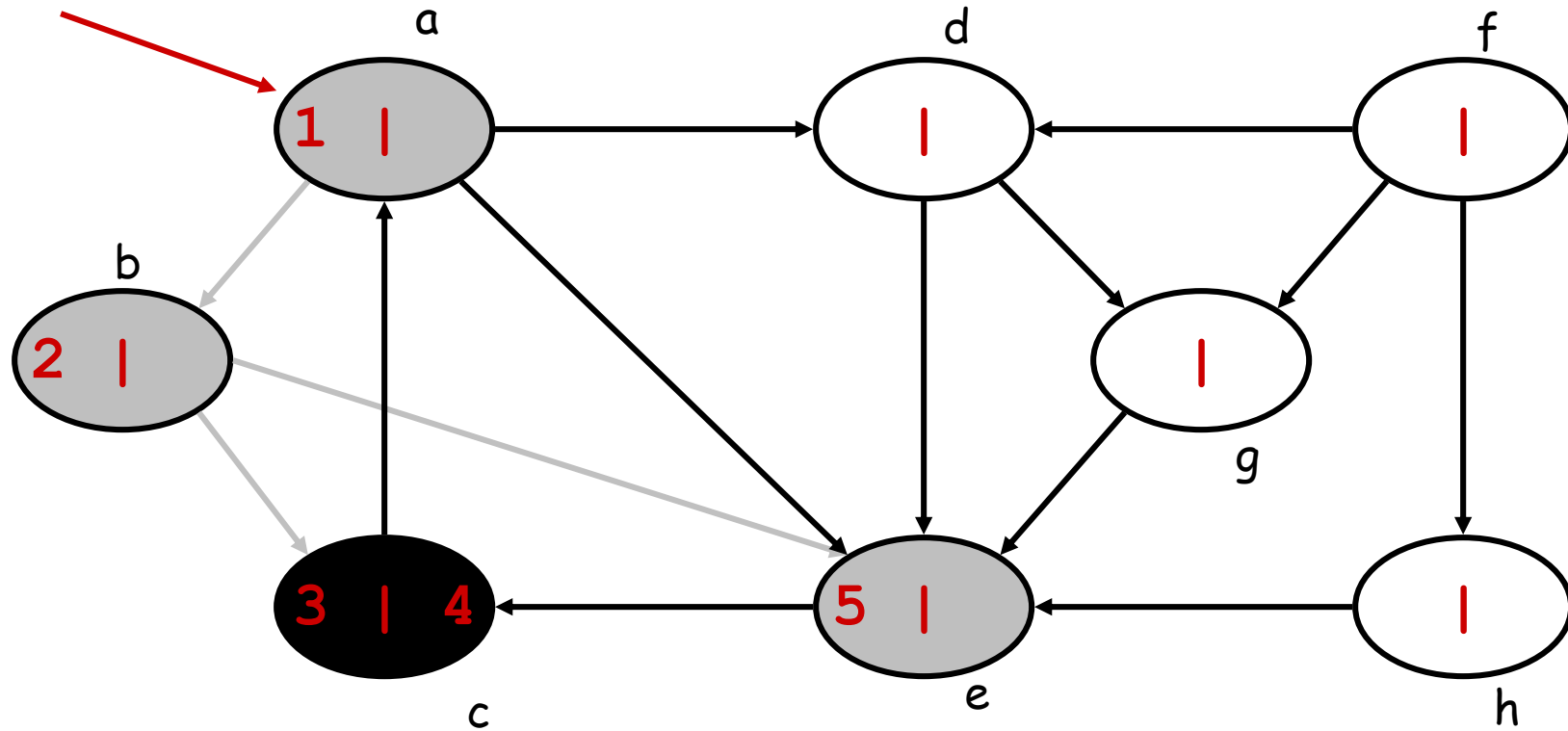
DFS Example

*source
vertex*



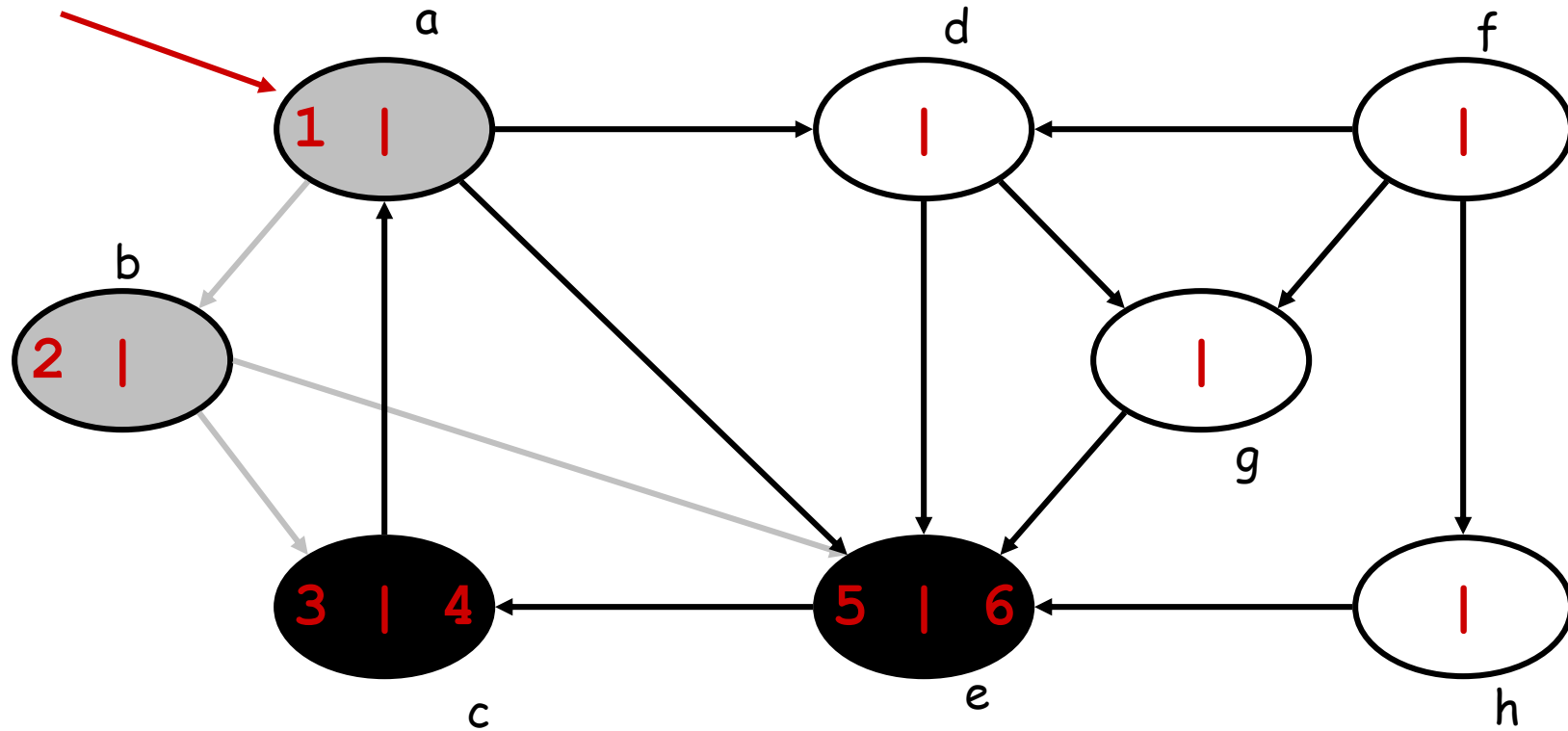
DFS Example

*source
vertex*



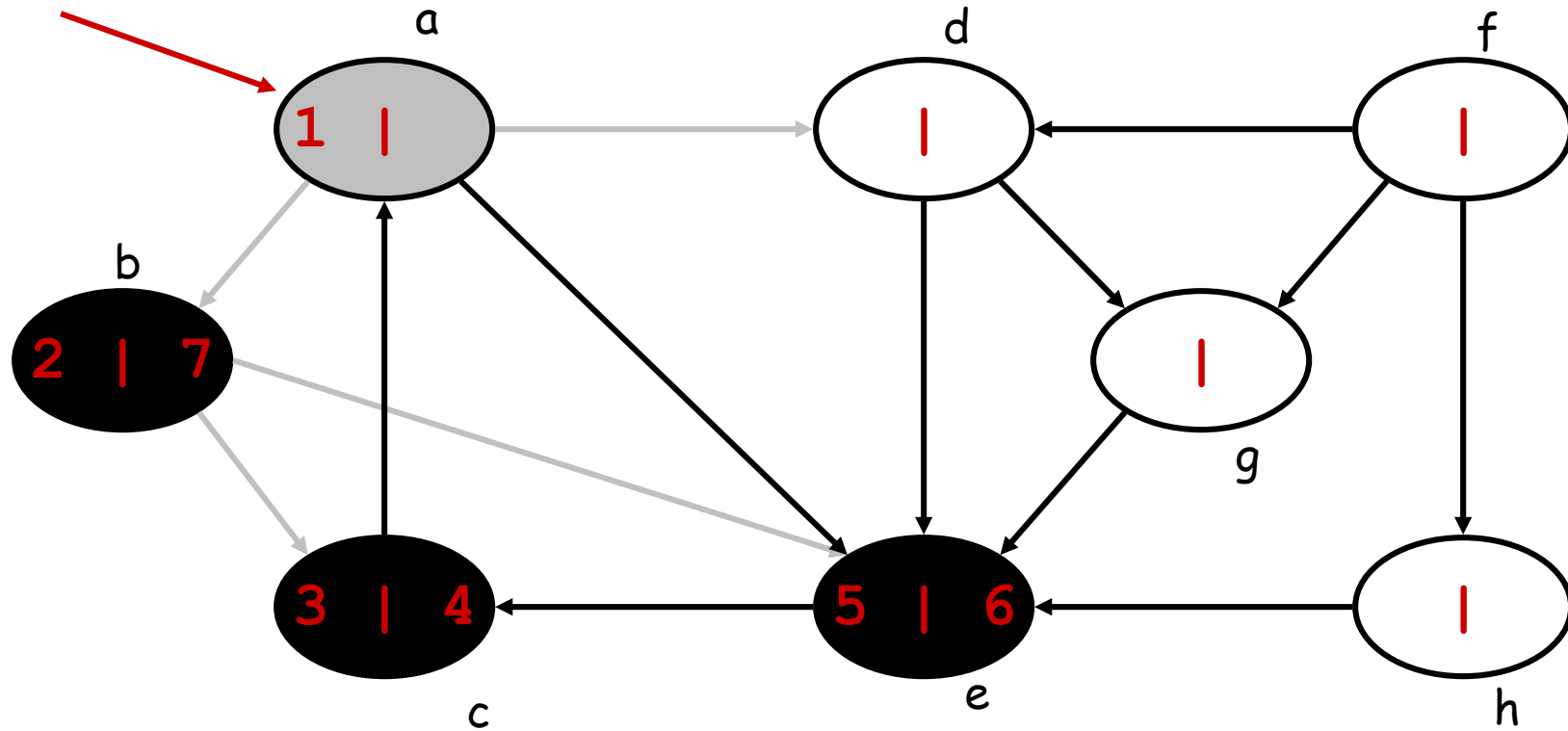
DFS Example

*source
vertex*



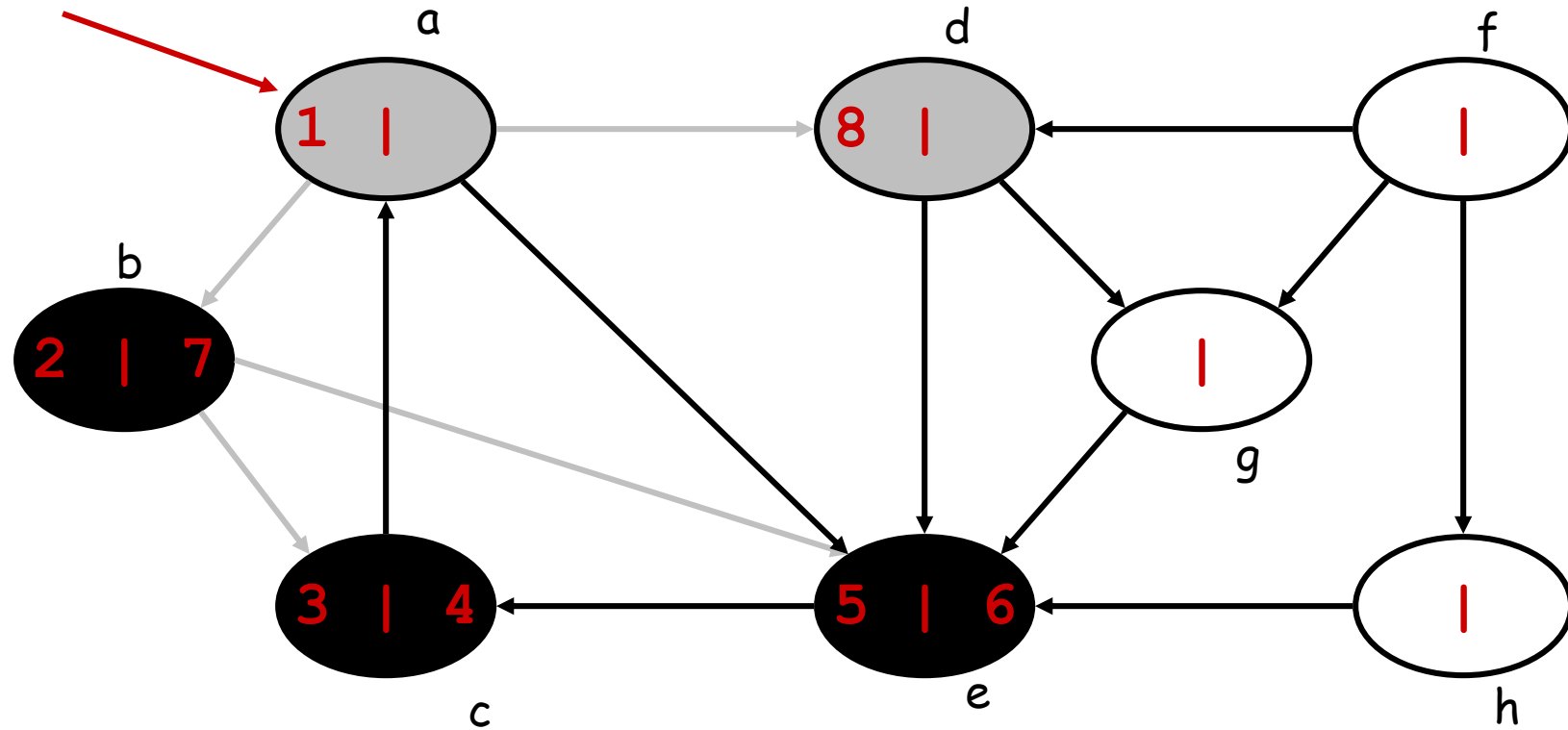
DFS Example

*source
vertex*



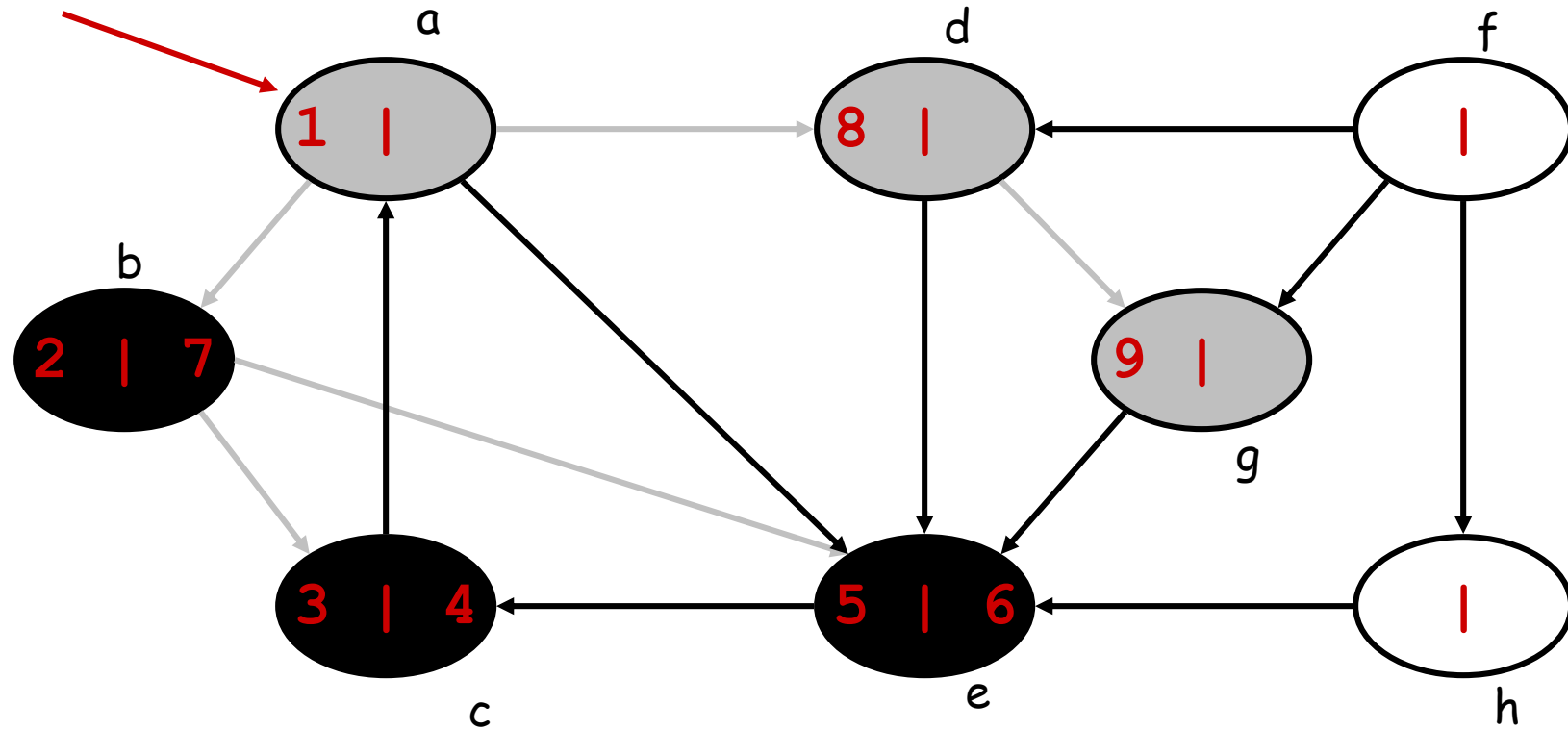
DFS Example

*source
vertex*



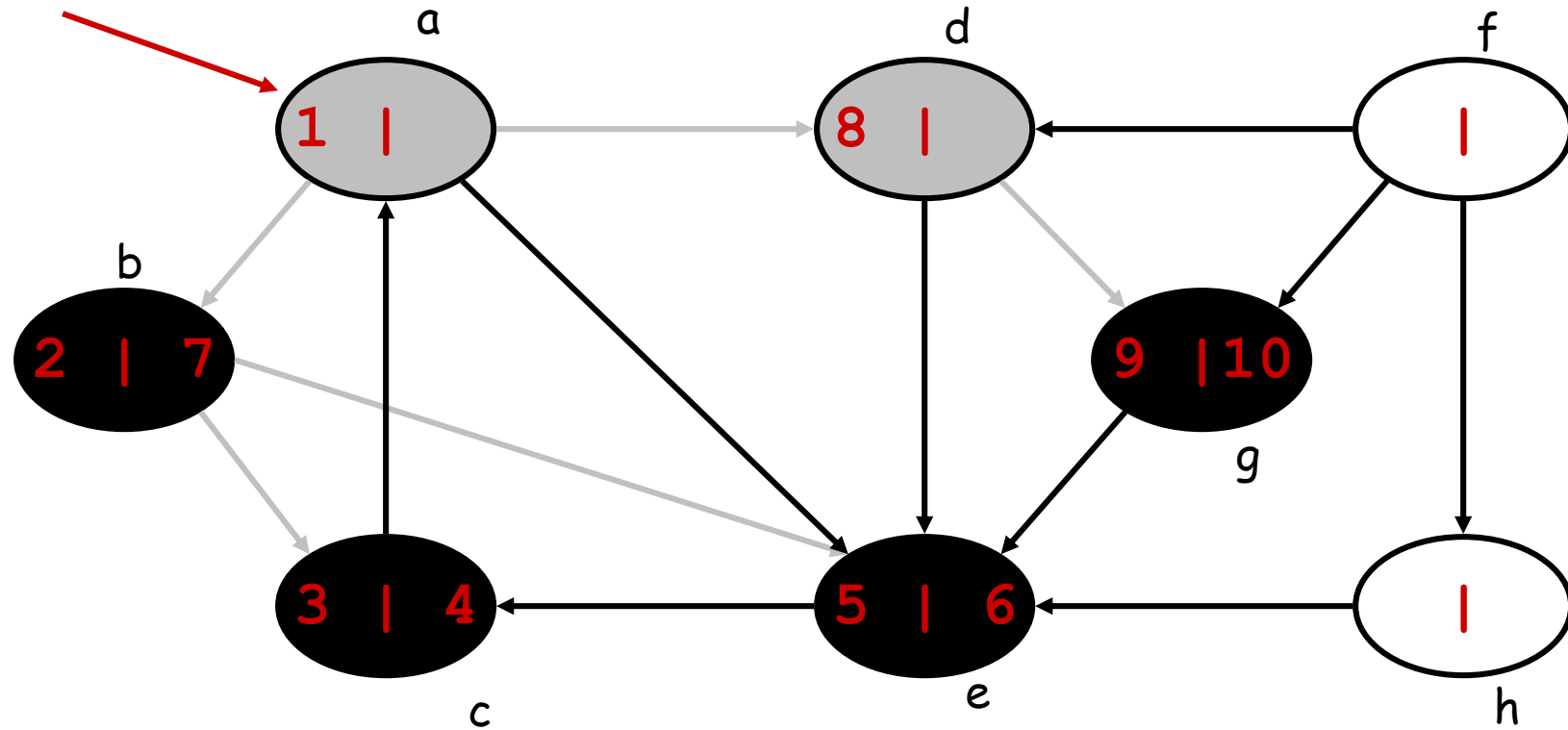
DFS Example

*source
vertex*



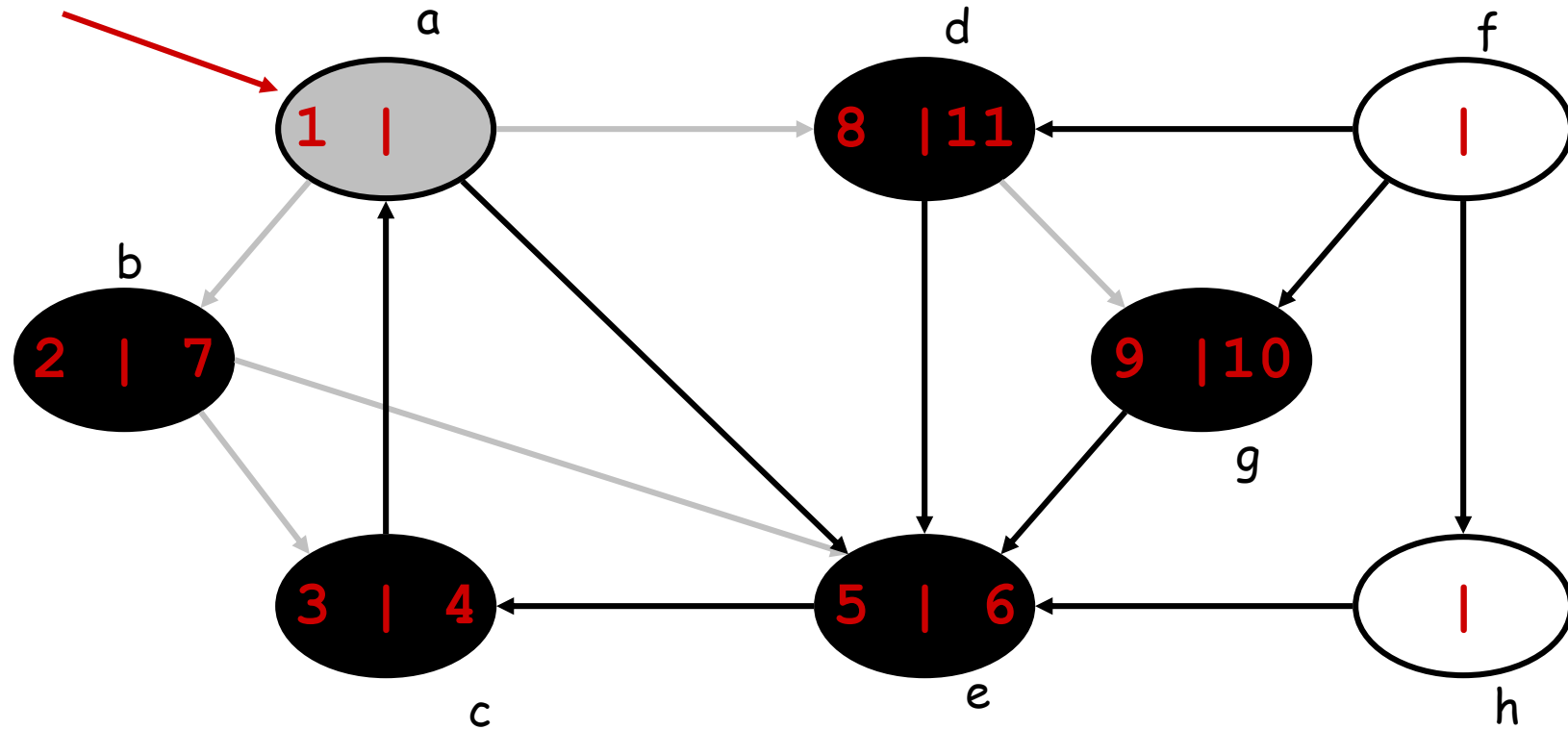
DFS Example

*source
vertex*



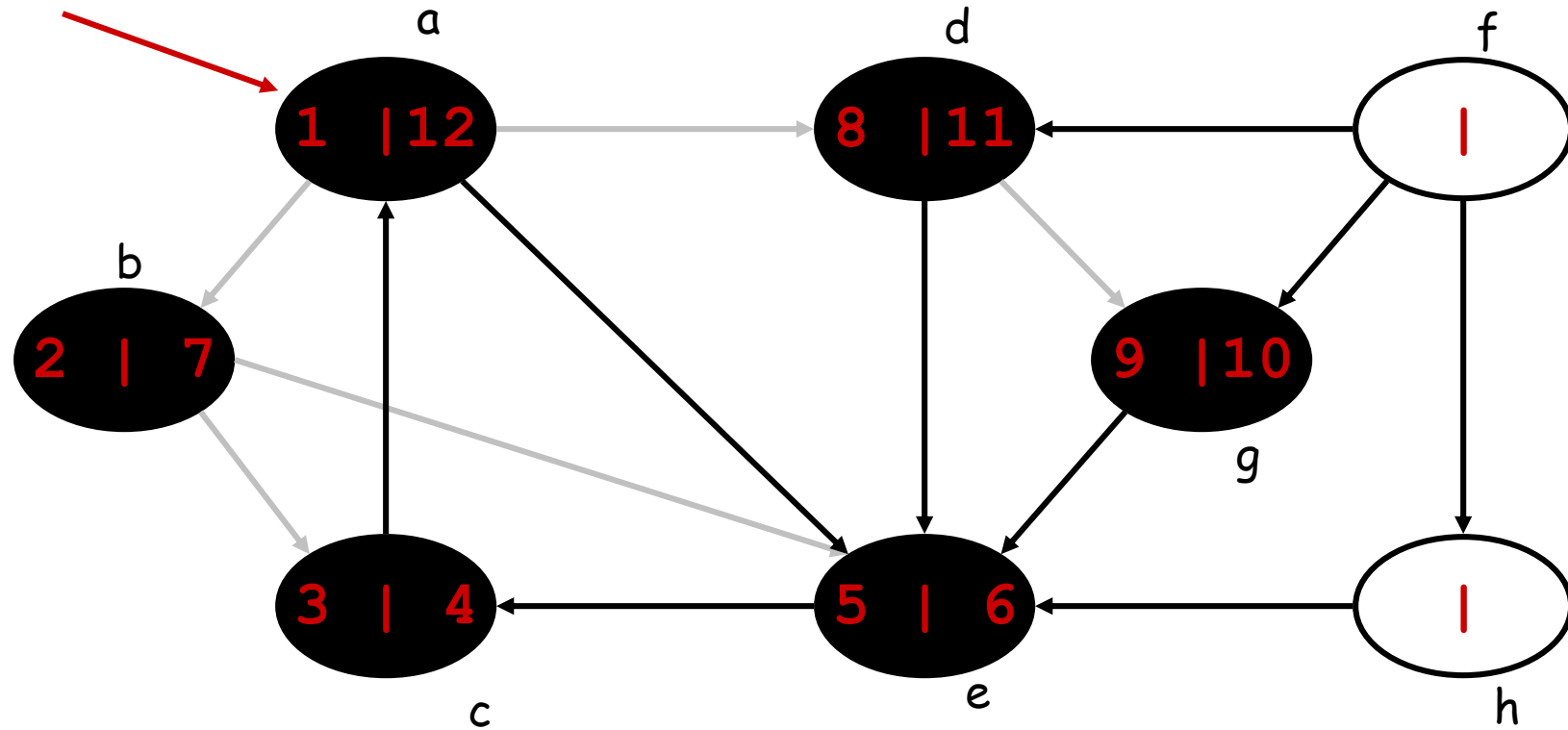
DFS Example

*source
vertex*



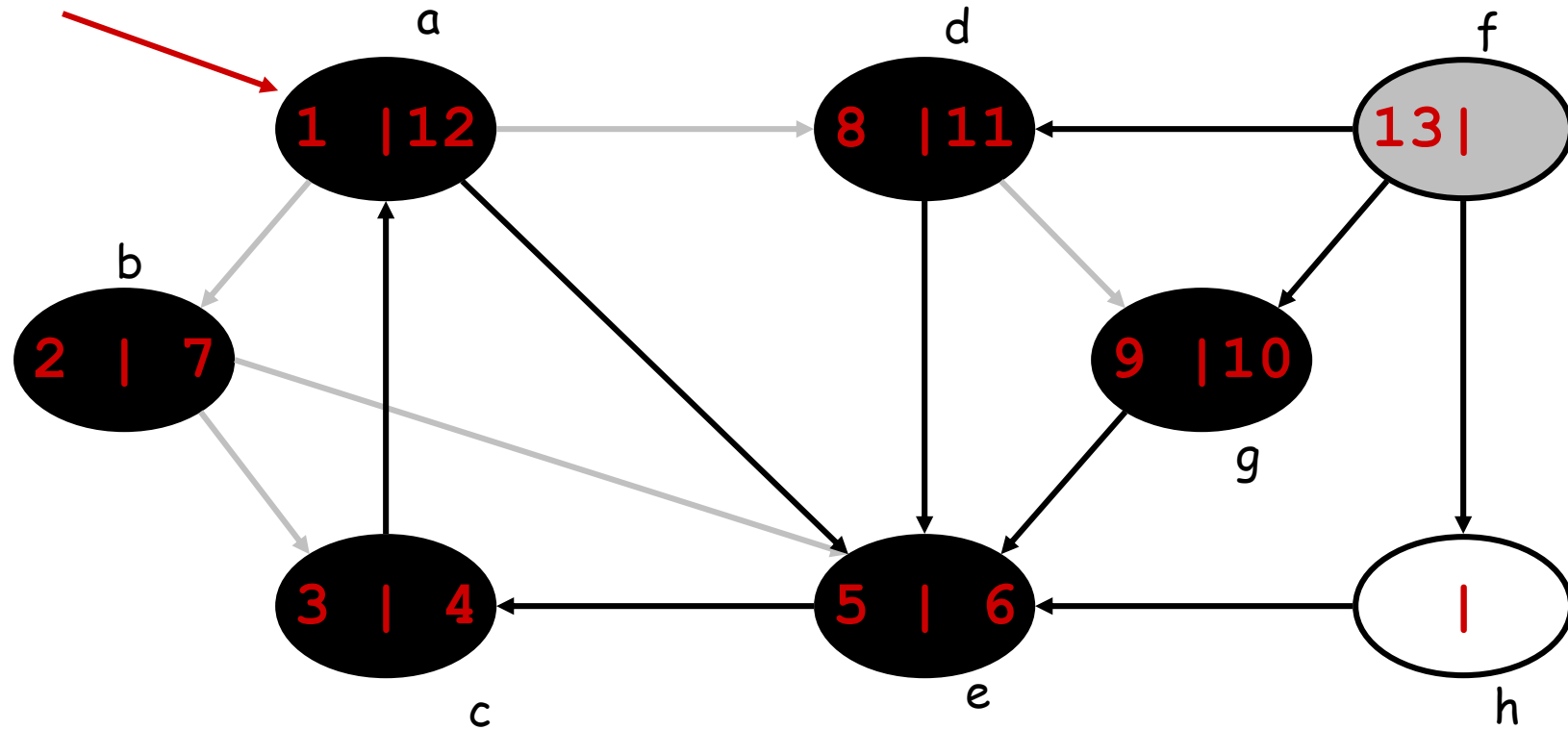
DFS Example

*source
vertex*



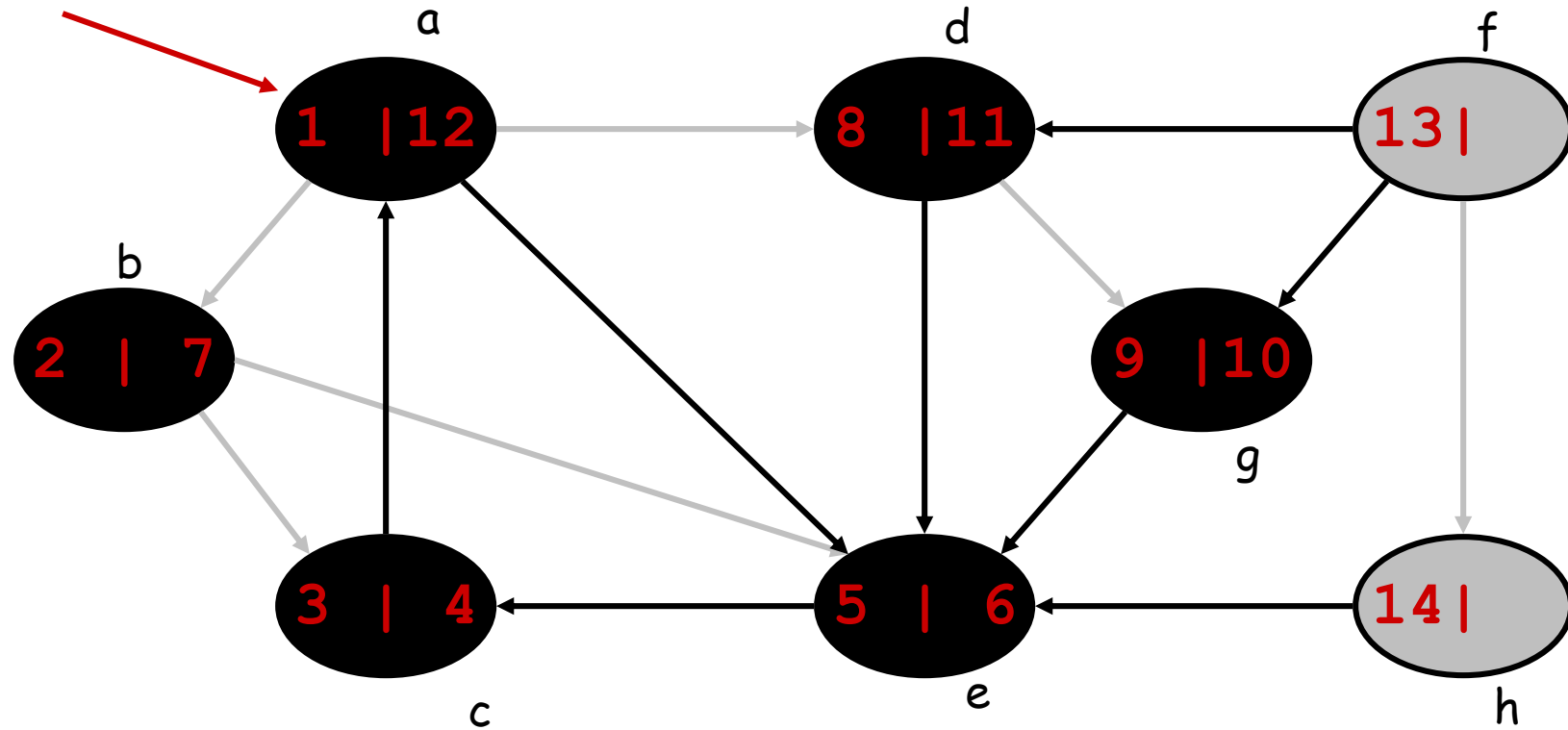
DFS Example

*source
vertex*



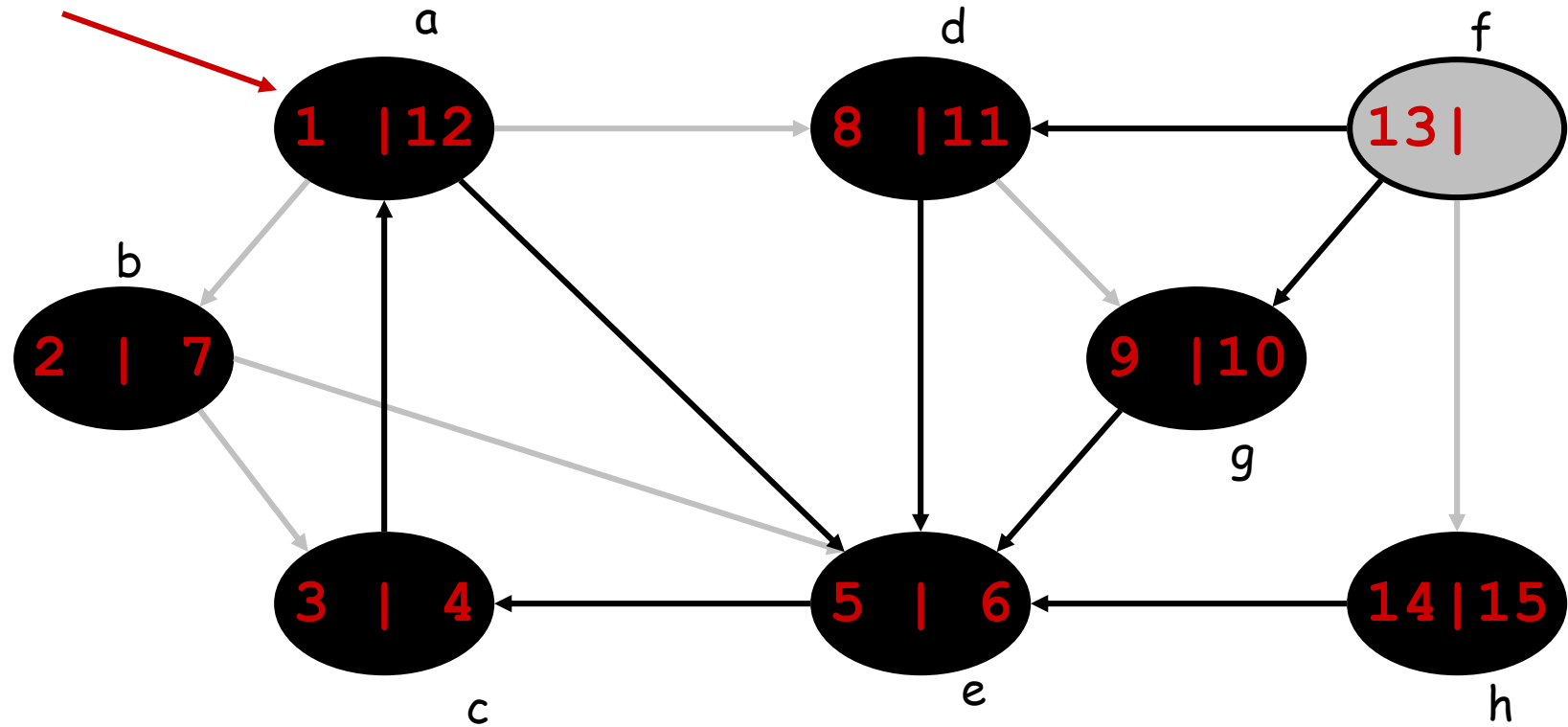
DFS Example

*source
vertex*



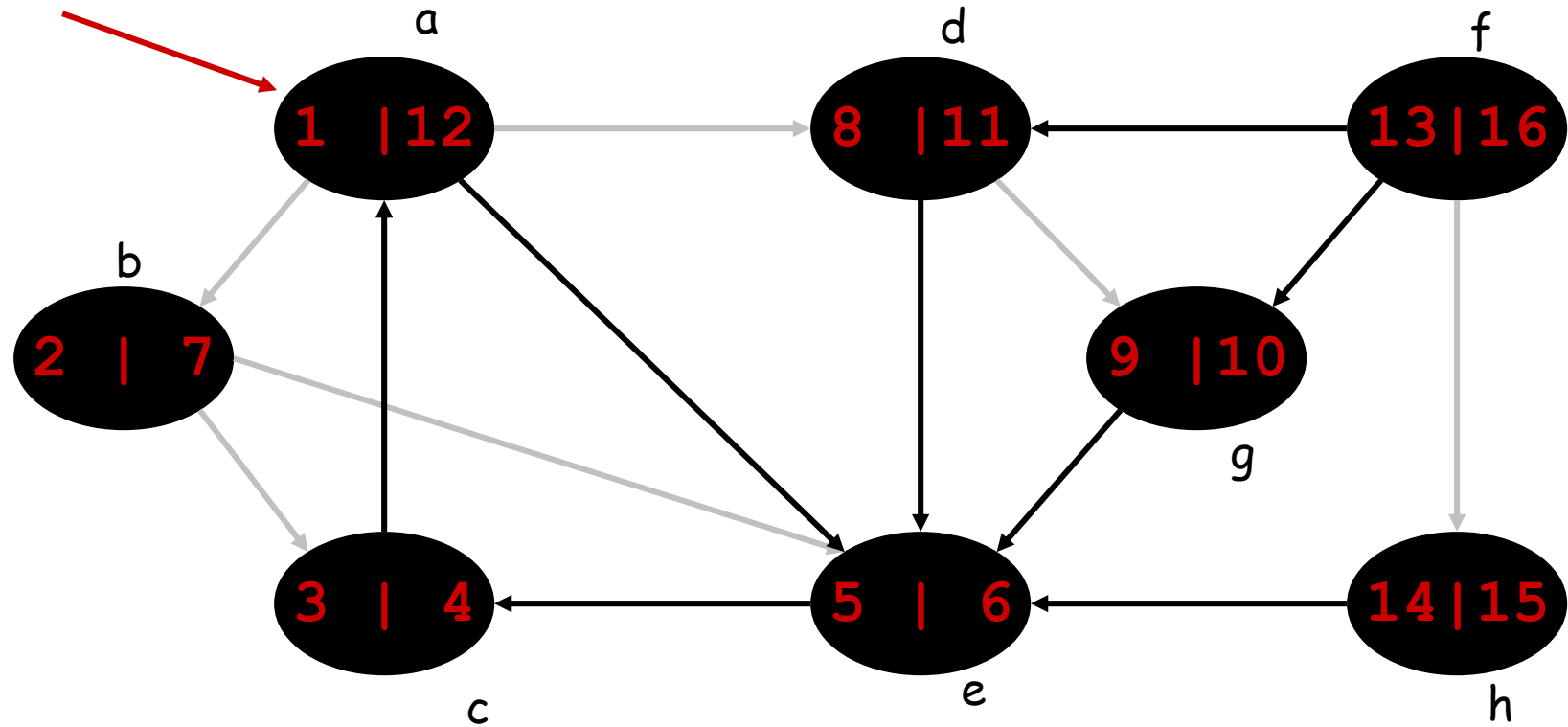
DFS Example

*source
vertex*



DFS Example

*source
vertex*



Running Time of DFS

initialization takes $O(V)$ time

second for loop in non-recursive wrapper considers each vertex, so $O(V)$ iterations

one recursive call is made for each vertex

in recursive call for vertex u , all its neighbors are checked; total time in all recursive calls is $O(E)$

Total time is $O(V+E)$